



Bachelor Thesis

Extending Interoperability: Type-Safe Bindings and Polymorphic-Interop Between Julia and C++

by

Kai Erik Niermann

(2720905)

First supervisor: Atze Van der Ploeg
Daily supervisor: Atze Van der Ploeg
Second reader: Verano Merino

June 12, 2025

*Submitted in partial fulfillment of the requirements for
the VU degree of Bachelor of Science in Computer Science*

CONTENTS

Contents	2
Abstract	1
1 Introduction	1
1.1 Background and Motivation	1
1.2 Objectives	2
1.2.1 Research questions	2
1.3 Scope and Limitations	2
1.4 Structure of the paper	2
2 Background and Tooling	2
2.1 Language Interoperability Foundations	2
2.1.1 Foreign Function Interface (FFI)	2
2.1.2 Application Binary Interface (ABI)	3
2.1.3 Practical Interop: C-ABI and Wasm	4
2.2 Interoperability Tool Architectures	5
2.2.1 Tool Design Patterns	5
2.2.2 Similar languages	6
2.2.3 Dissimilar languages	7
2.3 Memory Management Strategies	8
2.3.1 Static strategies	8
2.3.2 Dynamic strategies	9
2.3.3 Hybrid strategies	10
2.4 Type Interpretation Strategies	10
2.4.1 Static interpreting	11
2.4.2 Dynamic interpreting	11
3 Implementation and System Design	11
3.1 System Overview and Setup	12
3.1.1 Implementation approach	12
3.1.2 Choice of Binding Generators	12
3.1.3 Implementation setup	13
3.2 Challenges and Design Decisions	13
3.2.1 Problem Statement	13
3.2.2 Metaprogramming and Partial Evaluation	15
3.2.3 A Record Type Formalism	17
3.2.4 Case Study: Jluna's Internals	21
3.3 Solution and Language Extension	22
3.3.1 Abstract Types & Substitutability	23
3.3.2 Substitutability in Julia and C++	24
3.3.3 Representing Fields & Types	25
3.3.4 Semantics of Unboxing & Boxing	26
3.4 Evaluation	28
3.4.1 Performance and Optimizations	28
3.4.2 Developer Experience	28
3.4.3 User Experience	29
3.5 Limitations	30
3.5.1 Slow and varying generics support	30
3.5.2 Asymmetric metaprogramming features	30
3.5.3 Decentralized language evolution	30
3.5.4 Theoretical exploration vs practical needs	30
4 Conclusion	30

Extending Interoperability: Type-Safe Bindings and Polymorphic-Interop Between Julia and C++

Kai Erik Niermann

Vrije Universiteit Amsterdam
Amsterdam, The Netherlands
k.e.niermann@student.vu.nl

ABSTRACT

Your Master Thesis has to be written in English and should follow the ACM style (see the [Overleaf template](#)). We also suggest a *structured abstract* following the structure below.

Context. Write this at the end

Goal. Write this at the end

Method. Write this at the end

Results. Write this at the end

Conclusions. Write this at the end

As a very rough indication, the final thesis report typically entails, excluding appendixes, between 10 and 30 pages, with an average of 15 pages. The large variation depends on many variables including the specific field, project nature, and context. We advise to ask your supervisor if you should consider which number as a reference.

1 INTRODUCTION

1.1 Background and Motivation

Two or more pieces of software, such as languages following different design paradigms in their compilation approaches and linguistic properties, working together via some means of communication forms the basis of what most people would understand as software interoperability. Interoperability is no new concept; it ties quite naturally into how the software systems we interact with evolve. Abstraction has been central to the innovations in programming languages to align more intuitively with natural language and make them more efficient for programmers. As many new programming languages distanced themselves over time from the hardware they had to execute on, we had to develop systems that translate these composited abstractions, which define most software, back into the basic codes that machines can apply to execute said abstractions. Two notable byproducts of this type of evolution are that

- (1) Performance characteristics in kind evolved with the abstractions we developed to design new programming languages.
- (2) Not all abstractions are created equal, so naturally, languages evolve in various directions, built on various execution models.

A tradeoff between performance and ease-of-use can lead to a natural, albeit conflicting, question of how to balance ease-of-use abstractions and the desired performance characteristics of the granular low-level control of hardware. Introducing domain-specific or new language abstractions can additionally lead to questions about how language abstractions differ and in what ways these differences are more applicable to some domains than others. All these considerations- performance, ease of use, language-specific features, or, more abstractly, the unique and evolving qualities of programming languages- should then naturally lead us to why we

might want to bridge specific attributes to "get the best of both worlds."

A byproduct of the increasing number of programming languages, especially ones that improve upon older languages considerably in aspects such as memory safety and performance, has led to an increased effort to develop practical solutions for language interoperability both to create bridges to well-established legacy codebases and to gradually rewrite codebases to align with modern safety standards and performance needs. Accounting for this increase in programming languages, ahead-of-time (AOT) compiler designers generally adopted modular architecture to provide an extensible language development framework supporting various languages. Extensibility in AOT compilers typically follows the pattern of a front end, which a language developer must implement, some language-agnostic intermediate representation (IR) middle-end, such as GIMPLE for the GNU Compiler Collection (GCC) or LLVM-IR for the clang compiler, during which compiler and language developers can write generic or custom optimization passes, and a back-end during which the optimized IR is transformed into machine code to target some instruction set architecture (ISA). Separation of concerns: having a front, middle, and back-end with AOT compilation while providing flexibility can still be limited because we are confined to the implementation framework of the compiler. Interpreters address the constraints of AOT compilers by providing a mostly platform-agnostic runtime execution model that trades bare-metal performance for the increased flexibility of language design by abstracting away the difficulties of low-level considerations. Just-in-time (JIT) compilers aimed to leverage the runtime information gained from interpreted execution approaches and combine this with more traditional bare-metal compilation to have both runtime language dynamism and performance through paradigms such as Profile Guided Optimization (PGO) enabling specialized recompilation of *hot*, frequently executed, code segments. Combining various execution models: interpreted, hardware compiled with IR and custom byte-code, and implementing runtime optimizations, usually means JIT compilers have a rather complex and bespoke pipeline leading to difficulties in their adaptation to a broader set of languages outside of the main set they were designed around. Increasing the modularity of JIT compilers or more generally the virtual machines (VMs) they are part of has been an active area of research and has had considerable innovations both evolving VM ecosystem's such as the Java VM (JVM) from Oracle with Java, Kotlin, and Scala in addition to providing frameworks to implement new languages such as the GraalVM [34] or Microsofts Common Language Runtime (CLR) [43] C#, F# and Visual Basic .NET (VB.NET) with community projects for other languages like C++, JS, Python and so on.

Reconciling the distinct qualities of AOT-compiled languages such as C++ and a modern JIT-compiled language such as Julia in the context of interoperability is a primary motivator for this paper. Further considerations are not only the practical ramifications of bridging Julia and C++ but also the insights we can gain from attempting to understand a more unified picture of how we can bridge this, in some ways, diverged evolution of different features of programming languages, especially in a contemporary context. We will consider two angles in this research. To provide a foundation for understanding the research’s more practical and applied elements, we will elaborate on the background that underpins the most common form of interoperability and how this takes shape in modern interoperability tooling to contextualize the later case study into the broader landscape of the tooling. A grounded, more applied perspective will also be explored by performing a practical case study through the application and extension of an interoperability tool called Jluna for C++ and Julia. As both Julia and C++ have an overlap in being used in scientific and high-performance-computing (HPC) applications, this case study will specifically be centered around using C++ to implement a performance-intensive segment of a ray-tracer implementation in Julia mirroring how one might use existing well-established C++ libraries to accelerate performance-critical segments of a larger application written in a more dynamic modern language such as Julia.

1.2 Objectives

Primarily the objective of this work is a to present tool-based design and case study in applied contemporary language interoperability research with a focus on system design, practical implementation and language semantics. Integral to understanding most concepts one learns throughout their academic and professional career is building a mental structure for what defines the shape and various aspects of a specific domain. Interoperability, being an expansive domain, can then be difficult both to build a larger structure of and understand how it ties into some of the broader aspects of programming languages. A central takeaway we aim to establish here is that despite this expansive nature of the language interoperability domain and the seemingly irreconcilable attributes of divergent language and compiler design paradigms there is a structure which can be understood and leveraged to extend tooling to better fit modern programming language semantics. Decomposing our objective into research we aim to explore the following

1.2.1 Research questions.

- (1) What are the foundations of FFI-based tooling, and how can WebAssembly (WASM) innovate on C-ABI based FFI approaches ?
- (2) How do interoperability tools address architectural patterns, memory management, and type representation challenges, and how do these strategies vary based on language and runtime similarity?
- (3) How can we create a more formal and integrated understanding of interoperability tools ?
- (4) What are some of the ramifications of extending existing tooling to support polymorphism across AOT and JIT compiled languages ?

1.3 Scope and Limitations

FFI-based contemporary interoperability tools and related background and theory will be the main focus of the research. VM language implementation frameworks, such as GraalVM, which have appeared more recently will be mentioned though not thoroughly evaluated. Additionally we will constrain ourselves to mainly tooling which was developed in the past 10 years or so and is mainly aimed at practical industry applications as opposed to more niche use-cases. We will also elaborate on some necessary background to understand the basic technology this interop tooling is built on though with a focus on its application in a modern context and evaluating it alongside some potential modern successors. Theoretical formalisms will also be introduced; though primarily as a means of contextualizing some of the practical implementation considerations through a type theoretical lens; less so as a means of rigorous formal verification. Situating the extension case-study within the context of similar tooling and some more traditionally theoretical concepts is the predominant focus of this research.

1.4 Structure of the paper

We will start by introducing language interoperability foundations, practical considerations and some future facing technologies like WASM which could serve as new basis for modern tooling to assess RQ (1). Evaluating RQ (2) will occur in two parts first looking into some of the more general strategies and design patterns we see for languages of different degrees of overlap in their design and implementation; second focusing specifically on memory management strategies in the context of AOT, JIT and hybrid AOT and JIT compiled systems. How interoperability tools actually understand types they want to bridge and the relevance of reflection as a contrast to older approaches will be assessed for RQ (4). Finally RQ (3) and RQ (4) will be explored through the extension case-study by demonstrating a potentially more integrated understanding of binding generators and to evaluate the ramifications of encoding and implementing polymorphism between an AOT compiled language, C++, and a JIT compiled language, Julia.

2 BACKGROUND AND TOOLING

2.1 Language Interoperability Foundations

2.1.1 Foreign Function Interface (FFI). Motivated by a desire to utilize some code in a *guest-language*, say C, which doesn't exist in a *host-language*, such as Python for example we have FFI-libraries. FFI-libraries, as the name might suggest, are typically libraries written for a host language which provide an interface, i.e. some "user-friendly" abstraction, to utilize functions from a guest-language. Despite this common division of *guest* and *host* FFI-libraries can often have segments written in both the guest and host-language, though a commonly shared attribute and important attribute is nonetheless that these libraries, despite the bidirectional nature of how data might move, do not necessarily have bidirectional interfaces, their interfaces are often developed around the paradigm of using the guest code from the host code. Central to the development of FFI-libraries is their reliance on a stable underlying Application Binary Interface (ABI). As the communication between languages at runtime fundamentally tied to their compilation systems; and thus

the underlying ISAs we are compiling to; FFI-libraries are typically built around some common ABI. C-compilers, across platforms, have adopted a practically stable and consistent specification of how to emit code at the binary level which has led to the most common form of FFI-library being based around the C-ABI conventions. An implication of the choice to enable interoperability primarily through the stability afforded by the C-ABI and the low-level simplicity of C's semantics has led to many mainstream languages such as Julia [58], Rust [61] or Python [4] having basic FFI-library implementations as a part of their standard libraries or early language histories.

We can construct a minimal example of what it might look like to interact with an FFI-library as a programmer. We can write a small greet function written in C in a file called `mylib.c` which we compile to a shared object (`.so`) format. In Julia, Rust and Python we can then broadly describe the common steps for loading and dealing with the library `mylib.so` as follows

- (1) *Loading* - Load the `.so` library into your host language code.
- (2) *Preparation* Prepare your host language variables for the guest-language foreign function.
- (3) *Usage* - Declare and/or directly access the desired foreign function through the provided FFI.
- (4) *Handling* - Handle the return value and possible exceptions.

```

1 // mylib.c
2 #include <stdio.h>
3
4 void greet(const char* name) {
5     printf("Hello, %s!\n", name);
6 }

```

Listing 1: FFI Example in C

```

1 ccall(:greet, "mylib"), Cvoid, (Cstring,), "World")

```

Listing 2: FFI Example in Julia using the standard `ccall` function

```

1 #[link(name = "mylib")]
2 extern "C" { fn greet(name: *const c_char); }
3
4 fn main() {
5     let name = "World";
6     let c_name = CString::new(name).expect("CString::new failed");
7     unsafe {
8         greet(c_name.as_ptr());
9     }
10 }

```

Listing 3: FFI Example in Rust using `extern` and the `link` attribute for the library

```

1 mylib = ctypes.CDLL('./libmylib.so')
2
3 mylib.greet.argtypes = [ctypes.c_char_p]
4 mylib.greet.restype = None
5
6 name = b"World"
7 mylib.greet(name)

```

Listing 4: FFI Example in Python using the `ctypes` library

Although generally describing any piece of software which provides an abstracting for handling foreign code FFI-libraries are most commonly associated with specifically C-FFI libraries, which is to say interop tools in which C is the guest language. For the purposes of this paper we will consider FFI-libraries in the broadest possible sense so tools with arbitrary source and guest languages, albeit still generally built upon the C-ABI as the intermediary. In the examples of different languages interacting with C there are some important observations we can make, especially in regards the nature of the *preparation step*. Conformance to the C-ABI means that any pieces of data which pass into this binary code compiled for said specifications must in turn also conform to this specification. Coercing data to fit the C-ABI conventions leads to a kind of linguistic bottleneck where types which have language specific usage paradigms (i.e. structs with specific unique offsets or metadata) have to be deconstructed into a valid format to transfer across the language boundary and then uniquely reconstructed within our guest language if they want to mirror idiomatic usage or retain certain information. Deconstruction of types we saw with the preparation of translating language specific string formats into c-strings, reconstruction we will encounter later.

Function signatures are another important aspect of this reliance on the C-ABI. Similarly with parameters the signatures themselves can't natively; by which I mean without an valid abstractions; exist if they don't have some low-level analogue. Generics are a prominent example of this. Due to the fact that generics tend to rely on language specific approaches for mangling overloaded function names with type specific information to create unique signatures; even if two languages have generics and maybe even similar approaches for instantiation; a limiting factor will first and foremost always be the fact that the C-ABI, by itself, does not have any specification for mangling. Hence something like we see in listing 5, cannot exist without deviating from a pure C-ABI based FFI-library.

```

1 extern "C++" {
2     fn add<T>(a: T, b: T) -> T;
3 }

```

Listing 5: Pseudocode example of `extern C++` generics in Rust

While then providing a stable common ground to build FFI-libraries on, the C-ABI, regardless of how much of the preparation you are able to abstract with FFI-libraries, only forms a very minimal and often limiting basis in the development of interoperability tooling. Bridging modern language features, especially ones which aid in code reuse, requires various; often language specific; strategies to evolve these libraries from their basic set of bridgable information.

2.1.2 Application Binary Interface (ABI). An important question which arises, especially as a language grows, is how changes to its binary code generation process deviate from older ABI specifications and hence impact compatibility with source code compiled with implementations based on said specifications. To mitigate communication issues between binaries compiled following different ABI conventions compilers tend to adopt specific conventions for a certain set of compiler versions, generally called *partial ABI stability* as we do have a common convention for a set of compiler versions though no guarantees that this will continue indefinitely. *Full ABI*

stability or just *ABI stability* is in contrast colloquially understood to mean that from a certain compiler version onwards all application binaries will have standardized conventions in how they interact. Swift’s ABI is a recent example of reaching stability with it first introducing run-time stability in 2019 [50], enabling compatibility between binaries at runtime as long as they guested the stable ABI (where compiled with a swiftc version ≥ 5.0), and compile-time stability enabling developers to use precompiled frameworks within their source code for compiler versions past ≥ 5.1 [46].

Persistence of the C-ABI as the de-facto standard for implementing FFI-libraries can in large part be attributed to its stability and the legacy nature of C as a language. Despite the aforementioned limitations for more complex tasks; the C-ABI’s rather constrained and minimal specification; given its existing popularity the C-ABI served as an effective basis for bridging languages who’s compilers where capable of guesting its convention. Utilising often language specific features such as compile-time code generation (i.e. macros in Rust, or templates in C++) like we see with CXX for Rust $\leftrightarrow$ C++ [31] or leveraging a languages runtime models to introduce more dynamic behavior such as enabling runtime recompilation as we can the work of JavaCPP [5] has allowed developers to expand considerably on the more limiting core attributes of the C-ABI. Performance overhead nonetheless remains a rather fundamental aspect of working with C-ABI based FFI-libraries as most innovations still generally add more layers of indirection a variable or object must pass through. Effective memory management, specifically limiting unnecessary copies and performance intensive memory manipulations, has been a crucial mitigation strategy for FFI performance overhead, often coming in the form of wrapper types such as in the case of the Julia $\leftrightarrow$ Python library JuliaCall which innovates on an older library, PyCall, by never copying mutable objects across the language boundary by default [51]. Integrating more complex features into C-ABI FFI-libraries to bridge two languages we can then note tends to require a clear focus on balancing the level of runtime indirection and manipulation of objects and making sure to understand how to most effectively encode and transfer objects across a primarily C-based intermediary language pipeline.

2.1.3 Practical Interop: C-ABI and Wasm. Painting a more practical picture of what it means for an FFI-library to be C-ABI based we can introduce two languages our guest L_g and our host L_h . Central to most FFI implementations based around the C-ABI are usually two pieces of code, the libraries core, which generally consists of a dynamic library compiled to a platform specific C-ABI target and the guest API, which provides methods that allow accessing the dynamic library, usually built on more fundamental features provided by L_g ’s runtime or compilation model. Conformance to the C-ABI is what tends to form the basis of how the guest API is implemented. Resulting from this choice to enable your language to be compatible with C-ABI compliant binaries, such as our dynamic library, comes with the fundamental implication that you need to coerce your types into C-compatible formats as the contractual obligation in the guest API. Coersion of types; in this instance; into a C-compatible format is what is generally known as boxing. Boxing in the more general sense can be understood as the idea of abstracting away underlying properties of types to create a generic

”box” to contain them in. Programming language designers commonly use boxing in instances where the detail lost in abstraction is not necessary or wanted by the programmer in a particular instance a basic example being the `Number` class in Java. Boxing in the case of interoperability requires considerable nuance and often effort in how the Box is constructed, as the datatypes provided by C only provide the most fundamental constructs primitives, structs, etc. composing these to encode accurate sufficient information to utilize the type properly when unboxing is critical. Reconstructing the type in a way it can be effectively understood in the target language is what is generally understood by *unboxing*. Unboxing has been implemented using a large variety of approaches, as its generally the goal to be able to use a type idiomatically within the context of a language most design paradigms have been centered around balancing performance with idiomatic understanding of a type.

Finding the balance between performance and idiomatic representations of types tends to come down to how effectively the low-level encoded representation of a type from the guest language can be understood and applied in the target language and vv. Naturally this leads to languages whose design philosophies are closely aligned to that of C i.e. low-level performant systems based languages such as Rust, C++, Zig having a simpler path for exchanging similar types as their memory layout is similarly minimal. Conversely languages which deviate away both in execution model (JIT instead of AOT) and thus design paradigms (flexibility over performance) tend to be more complex when working with types which exhibit boxing; metadata and a more complex memory layout; within the language. Nonetheless we have seen both languages focused on dynamism exist that borrow from AOT languages such as Julia with its integration of LLVM-IR into its JIT compilation pipeline and strong tie-ins with C via its C-FFI library, and languages focused on performance such as a Rust and C++ integrate features which enable more compile-time and runtime dynamism such as Rusts macros system or C++ compile-time code generation via template meta-programming. Despite these innovations in programming languages, and the corresponding adaptations made to C-ABI based FFI-lib to bridge modern features, the increasing complexities of language semantics and execution models poses a valid question for the extent to which the C-ABI should continue to be utilized in the development of modern interoperability tooling.

Contrasting the C-ABIs predominantly hardware based nature a commonality we can observe with various VM-based runtime ecosystems is that as they mature, to provide the necessary extensibility to incorporate new languages, they tend to implement a platform-independent intermediate code called bytecode. Bytecode is typically executed at runtime via an interpreter as a part of a VM or compiled via an implementation of JIT compilation. Some nuance exists in the terminology as we do have some unique instances such as with Julia which, while JIT compiled, does not utilize bytecode, it instead opts for the more traditional approach of using just LLVM-IR which it compiles to native code [59]. Two of the most widely targeted instances of bytecode are JVM-bytecode via the GraalVM [32] and *WebAssembly* (Wasm). First announced in 2015 and having its design ratified in 2017 [71]; Wasm is a bytecode format designed to be executed within web-browsers using stack-based VMs. Despite the initial runtime target of stack-based VMs; more generally the browser; nowadays there are also AOT

compilers implemented to compile Wasm to native code such as WasmEdge [70] or JIT compilation implementations such as Graal-Wasm [33]. Some instances of the most common and important compiler toolchains for Wasm can be seen in Table 1.

Emscripten was an early toolchain that enabled compiling native C and C++ code to Wasm, providing a virtualized POSIX-like environment for programmers. This means that, at the API level, integrating C or C++ code with Emscripten allows native features such as file system access to use interfaces similar to those found in a standard POSIX/Linux API, requiring minimal changes to existing native code [25]. The use of a virtual POSIX-like layer is primarily motivated by the expectation that Wasm binaries will be executed within JavaScript or Wasm-supported VMs, such as browsers or Node.js. However, the close coupling between the compiled .wasm code, its runtime, and the ABI specification—as seen with Emscripten—can become problematic for server-side¹ Wasm runtimes. This is because Emscripten’s rapid and flexible development of POSIX emulation, while beneficial for browser compatibility, can hinder the stability and broader adoption of its binary specifications [2], [3]. In other words, the flexibility that allows Emscripten to quickly improve POSIX emulation also makes it less suitable as a stable foundation for server-side or long-term ABI standards.

Aiming to address the issues of ABI stability and wider adoption of Wasm into server-sided runtime models the WASI subgroup has developed and continues to work on a group of standards under the WASI name [69]. A major focus by WASI and the Bytecode Alliance in simplifying interoperability and reaching ABI stability has been the introduction of the Canonical ABI. Similarly to the C-ABI the Canonical ABI aims to standardize how data moves across language boundaries. To standardize interoperability with the Canonical ABI the approach taken is to use an *Interface Definition Language* (IDL) called *Wasm Interface Type* (WIT). WIT is used to define specifically what features a library offers, this is segmented into 3 levels of abstraction, an *Interface* which represents types and functions, a *World* representing a group of interfaces and a *Package* representing a set of WIT files also known as *Components*. Binary compatibility is ensured through an interface-centric design approach. Language-specific tools generate bindings from your WIT code, creating a scaffold for your implementation. When compiled with a Canonical ABI-compliant toolchain, this ensures compatibility with any other Wasm binary compiled the same way, regardless of the source language [11].

Interoperability facilitated through the Component Model and its underlying Canonical ABI has a few benefits over C-ABI based approaches. Since compatibility between binaries is ensured by the nature of having an ABI compliant implementation scaffold, no boxing or unboxing between types has to occur. Canonical-ABI compliant toolchains in essence offload the necessity of boxing and unboxing as ABI compliant interface encodings are an inherent property of the generated bindings. The obvious drawback of this approach is that existing libraries which utilize language specific features not representable via WIT do require some compliant WIT-encoded representation or risk incompatibilities, though unlike the

C-ABI which on the API level is based on C’s type model, WIT models a much wider range of modern language features as a part of its core specification then could be trivially encoded and compiled following the C-ABIs standards. WIT is designed from the ground up as an IDL, specifically for modeling interfaces. These interfaces can then be used to generate ABI-compliant bindings. This makes WIT a strong candidate to replace C’s type system as the de facto standard for interface specification in traditional FFI libraries.

2.2 Interoperability Tool Architectures

2.2.1 Tool Design Patterns. Three important attributes shared amongst most of the popular interoperability tools, *Mediation*, *Validation*, and *Generation*. Mediation is what tends to exist at the core of most implementations, it refers to the approach which is used to reconcile semantic differences across language boundaries. Earlier, in section 2.1, we discussed the C-ABI as a popular choice to act as the common ground in the mediation process. A more recent alternative to ABI compatibility as the common language has been API compatibility. API compatibility is generally more popular in managed languages, i.e. JIT compiled or interpreted, which integrate APIs into their runtimes which allow extending a language with native modules that comply with the APIs high-level conventions. The primary difference between interoperability at the API and ABI level is how compatibility is ensured. Compatibility at the API, or source-code, level is enforced through high-level language constructs which define a contract for how to interact with some natively compiled binary. At the ABI level compatibility is enforced by languages following a common platform-specific ABI specification. Though to clarify ABI and API compatibility are not mutually exclusive, when an interpreted or JIT compiled language calls API compliant native code, ABI compatibility must still be ensured, the idea being that if a piece of native code is compliant with a languages API, then safe, or ABI compliant, interaction should be guaranteed through the implemented higher-level constraints.

Generation and Validation are typically the guiding factors in implementing safe API compliance. In practice what API compliance actually looks like is the bridge from the code in your guest language to idiomatic code in your host language. Code which is API compliant should typically behave according to the hosts languages semantics, which is to say in an idiomatic fashion. Bridging code occurs most often through translating guest language interfaces into ones which adopt host language semantics. The tools which interpret and generate compliant or valid interfaces, also known as bindings, to enable a safe exchange through some mediation layer, are aptly named *binding generators*. Binding generators can be generally classed into two subtypes, ones aimed at creating API compliant bindings, and ones aimed at creating ABI compliant bindings. Generating API compliant bindings means the implementation tends to be focused on translating guest language interfaces into something which the host language API can interpret. On the other hand generating ABI compliant bindings revolves primarily around ensuring that the shared generated interfaces on both languages, once they are compiled, have compatibility on the binary level. What differentiates ABI compliant bindings from API compliant ones is the specification of the mediation strategy they are built around. API compliant bindings use the specification of the native

¹Server-sided runtimes in this case are not to be confused with the traditional server-client model of code execution. Wasm code executing on the server-side generally just means “outside of the browser or other sandboxed runtime”.

Compiler/Toolchain	Description
Emscripten	Emscripten is a toolchain for compiling C/C++ code to Wasm. It introduced a POSIX-like API for Wasm, though this ABI has not been formally stabilized. [73]
Cheerp	Cheerp supports compilation to both WebAssembly and JavaScript, facilitating seamless interoperability with web environments. It also integrates with ts2cpp, a utility that generates C++ interfaces from TypeScript types [24], [49].
WASI	The <i>Wasm System Interface</i> (WASI) subgroup [69] aims to define a stable Wasm-ABI for native interfaces. In conjunction with the standards defined by the WASI subgroup the Bytecode Alliance in conjunction with various contributors has been working on various tooling [8]. • wasienv: A higher-level toolchain supporting languages like Swift and C++ [3]. • wasi-sdk: A lower-level compiler toolkit requiring language-specific frontends [29]. • wasmtime: A standalone Wasm runtime implementing WASI [10]. • wasix: A compatibility layer that extends WASI with POSIX-like features [55]. • wasm32-wasi: A Rust-targeting WASI frontend integrated with the cargo build system [9]. WASI currently offers one of the most comprehensive and actively documented ABI models for Wasm-based systems programming.
GoJS / TinyGo	TinyGo is a lightweight compiler designed for resource-constrained environments, capable of compiling Go code to Wasm. Its documentation and ecosystem are still evolving. [6]

Table 1: Overview of Major WebAssembly Compiler Toolchains

API from the guest language to create host language native API wrappers around guest language interfaces and data. An intermediary language is typically what forms the de-facto API specification for ABI bindings. The byproduct of relying on an intermediate language, assuming that neither the guest or the host is also the intermediary, then naturally relies on both languages needing to have compliant interfaces to allow for an exchange. In practice the difference in tooling built around API vs ABI compliant binding generation often comes down to if a language already has an API for integrating native code or not.

Validation, which is to say actually ensuring compliance of the generated bindings can be separated into build-time and run-time verification. At build-time verification is generally implemented through either translations or intermediate wrappers which are based on either the guest languages native API or the API of the underlying ABI we are targeting, e.g. C’s interface and type semantics for the C-ABI. Build-time verification is primarily focused on establishing type-safe mediation mechanisms through conformance to the respective specifications. Run-time verification refers to approaches of validating messages across the language boundary, primarily to address language safety features (memory management, type checking, etc.) which are lost when using features from a guest language interface. To evaluate RQ (2), concerning the impacts of different language dynamics, such as a memory management approaches, on the design patterns in interoperability tooling we will take a look at both tools for languages with similar paradigms and ones aimed at more distinct domains.

Syntactically speaking, aside from very esoteric or functional languages (e.g. brainfuck, Haskell), programming languages roughly follow similar patterns in their design, i.e. loops, functions, data structures, variables. Semantically “similarity” is a more nuanced point to consider. We can narrow down the semantic similarity of languages if we understand it as the behaviors a language exposes to move data throughout its execution. We can break this down into languages which are focused on imperative procedures, exposing granular tools for manipulating memory, typically described as

low-level or systems languages, as they commonly exists close to bare-metal machine code. Languages which are based primarily on declarative, or simply high-level constructs, which abstract common instances of imperative control flow then tend to form the other semantic group. Strictness of typing is another point which can be used to delineate programming language semantics, but for our purposes we will use the perspective of memory management during control flow as we think it can provide some better insights in the context of evaluating RQ (2). Nowadays we do often see, especially for more mature languages, various back-end targets (JIT compilation, machine-code, etc.) which in turn leads to many languages often having features which deviate from its initial semantic focus, nonetheless the core design focus still tends to define the most common ways a language is used.

2.2.2 Similar languages. To evaluate tools aimed at similar languages we will use two pairs of languages, (C++, Rust) and (Julia, Python). Tools which revolve around bridging Rust and C++ most commonly position C++ as the guest language with Rust being the host language. Interoperability tooling developed around languages with low-level dynamics tends to exist primarily for the purposes of reusing or rewriting libraries from an older low-level language with a newer one that has similarly good performance capabilities but innovates in areas of type and memory safety. A relevant example of this has been the different tools developed to integrate and rewrite C++ libraries within the Rust ecosystem. Driving the move towards Rust are both internal, features of the language, and related external factors, government measures [45] and industry investment [53]. Due to the fact that minimizing overhead is a core tenant of both Rust and C++ neither of their compilers come with built in API-compliance. Because of the established nature of legacy C codebases, what the compilers for Rust and C do have is ways to compile C-interfaces and then produce compatible binary code via the C-ABI standard. Rust-centric C++ interop tools like autocxx [56], cxx [64] or zngur [39] all fundamentally rely on Rust’s and C++’s feature of declaring and compiling C compatible interfaces as the mediation strategy. The implementation for generation and

validation are largely dependent on how different people answer the question of how to parse and apply header files. Bindgen [63] uses libclang to consume C and C++ header files, which yields an Abstract Syntax Tree (AST) representation of the provided interfaces [14]. Using the AST representation of the interface, bindgen then generates Rust source code utilizing its C-ideological. Autocxx, builds ontop of a fork of bindgen and cxx, it uses the parsed headers from bindgen to generate the custom C++-IDL used in cxx. From the generated C++-IDL it relies on cxx to generate the underling C-IDL representing the C-ABI compatible glue code. The primary benefit of integrating the generation and parsing from bindgen with the C++-IDL from cxx is that it combines automated header parsing with automated IDL generation, essentially removing all burden on the programmer to write any glue code [21]. The IDL provided by cxx and its corresponding transpiling mechanism from the C++-IDL to the C-IDL also provides a more idiomatic and safe way to work with declaring and interacting with C++ interfaces in Rust then older more direct generation mechanisms [57]. Zngur differentiates itself from cxx and by extension autocxx by not embedding its C++-IDL within Rust, instead choosing to have it be specified externally in a .zng file [40]. The approach Zngur takes parallels the interface driven design we discussed earlier with WIT. While most popular interop tooling is focused more prominently on enabling cross-language dynamics through interfaces, emulating the traditional style of importing logically separated components, we do also see tooling which aims to not just have embedded cross-language IDL's but embedding foreign code itself. An instance of a language-embedding tool for Rust and C++ is the cpp crate [28]. Despite still being reliant on the common C-IDL and using macro expansion to automatically generate the compliant interface definition, it takes the unique approach of allowing inline C++ code within your Rust source. At build-time the cpp crate uses macro expansion and parsing of the inline C++ code to generate a static C++ lib consisting of the inline code which it links to the final Rust binary.

The use of compile-time code parsing and generation to implement the underlying binding generation, or even generating entire static libraries from foreign inline code, is a common pattern amongst these tools, motivated in large part by the goal of creating zero cost abstractions as both languages are generally applied in performance intensive domains. Having inline foreign source code draws some parallels to the approaches at interoperability through interpreter composition by the work of Barrett et. al. [7], a key distinction is how the approaches optimize crossing the language boundary. Barrett employs a technique called meta-tracing; tracing by itself refers to the idea of using information at runtime to evaluate hot code, optimizing it (e.g. through specialization), and compiling it to machine code. Meta-tracing focuses on tracing the execution of an interpreter instead. A meta-tracing JIT compiler is then a JIT compiler which, as opposed to taking in the source code of a particular languages takes as an input the syntax of an interpreter, the interpreter then in turn is what executes a given source code and while doing so has its execution traced and optimized by the underling JIT compiler. A benefit of using a meta-tracing JIT compiler in this case is that, as opposed to the cpp crates language composition, we are able to use runtime information to optimize objects crossing the language boundary. Zero cost abstractions

through compile-time code generation are an effective optimization technique, but limited in their optimization capacity as long as no run-time information is used.

Tooling for Julia and Python expands considerably in a more naturalistic composition of languages. For brevity we will focus on tools in which Julia is the host language and Python is the guest language, though various design patterns are naturally mirrored. To enable interoperability both Julia and Python fundamentally rely on native language APIs. Julia, being executed by a JIT compiler written in C, naturally has a low-level C representation of objects [60]. Similarly Python; running via the CPython VM; has a C-API which allows extending the language with low-level modules conforming to its API specification. Once again we therefore see C forming the common ground here, a main difference towards (C++, Rust) tooling being the application of language specific native APIs to wrap objects, as opposed to transpiling towards the compatible intermediate C-IDL. As Python is a language built around dynamism and thus heavily managed its object representations generally have sufficient metadata for easy parsing via Julia-sided code and then wrapping the object in a mirrored interface which holds on to the underlying data, generally avoiding making any copies. Both JuliaCall [51] and the earlier PyCall [37] follow the pattern of using low-level objects parsing to create interfaces which can attach to Python's C API object representation. To call back into Python code the JIT compiler implicitly uses Julia's ccall mechanism which in turn interacts with Python's C-API in essence replicating any other C-based module extension from the perspective of Python's VM.

2.2.3 Dissimilar languages. To assess the design patterns for tooling aimed at dissimilar languages we will look at tooling which bridges Julia and C++. As discussed in section 2.2.2 Julia provides a C-API for low-level language extensions. Similarly for the case of bridging Python over to Julia, Julia's C-API is used as the target for wrapping code. A common inspiration we can note amongst modern tools bridging C++ to a higher level language such as Julia for the case of CxxWrap.jl [22] or Python in the case of pybind11 [35] is the Boost.Python library [1]. The Boost C++ ecosystem are a set of libraries which are well known for their application of C++ templates and macros, in essence compile-time code generation. Boost.Python, following the Boost paradigm, uses compile-time code generation to create the compatible Python C-API shims², or adapter functions, to integrate with C++ code. CxxWrap.jl follows a similar principle. On the userfacing end it provides a set of macros and functions which can be used to register C++ classes and functions. During compilation the back-end macro's then translate the interface declarations into a dynamic library which exposes the registered functionality through a combination of Julia's C-API and custom generated shims.

Whereas CxxWrap.jl is focused on the more traditional guest-host language relationship, Cxx.jl [38] aims to achieve inline C++ code execution within Julia source code. The approach at inline foreign code execution we see in Cxx.jl differentiates itself from the cpp crate in that we don't separate out the languages at build-time and then execute them through an intermediary. Instead Cxx.jl execute the cpp code fundamentally through Julia's llvmlcall and

²The most common type of shim in interoperability are thunks which adapt arguments to fit a particular calling convention and then trampoline to the actual function.

the application of *staged functions*. Staged functions, which are declared using the `@generated` macro in Julia, are functions which implement the JIT analogue to overload resolution through compile time specialization. Julia’s JIT compiler, before it generates the machine code, will first evaluate the types of the staged function’s arguments, and then generate the machine code conditionally based on the types of the arguments. Cxx.jl uses staged functions to select the correct C++ function to call, in essence specializing the C++ function using Julia’s JIT compiler. Once inline C++ source has associated types, Cxx.jl uses clang’s frontend to generate the AST and from this emit the LLVM-IR. Using the LLVM-IR Cxx.jl then relegates the actual execution of the emitted IR back to the Julia’s built-in `llvmcall` mechanism. Contrasting the approach taken in Cxx.jl, we can see that it, to a degree, bypass the needed for certain optimizations either via meta-tracing or zero-cost abstractions, by simply emulating the native compilation flow of the target language i.e. compile-time specialization, clang AST generated, and injected LLVM-IR execution. Cxx.jl’s approach can be compared to that of the Python-C++ tool `cppyy` [41]. Both `cppyy` and Cxx.jl are able to execute inline C++ code efficiently by embedding different compilation models into the host languages runtimes. Cxx.jl implements a de-facto C++ JIT compiler by embedding parts of Clang’s frontend to parse and semantically analyze C++ code, and using Julia’s LLVM backend (via `llvmcall`) to compile and inline the resulting machine code into Julia methods. `Cppyy`, on the other hand, embeds the C++ interpreter `cling` [67] to similarly execute the inline source in a way which idiomatically aligns with the underling runtime of the host language, i.e. the CPython VM.

Jluna [18], while superficially similar to CxxWrap.jl, takes a different approach in how it works with the Julia C-API and templates to generate the necessary bindings. CxxWrap.jl, despite using macros, still relies heavily on dynamically allocated objects to generate the necessary glue code. Jluna, on the other hand, uses an almost exclusively static approach. Jluna utilizes the static information from the declared C++ classes and functions to conditionally compile or specialize meta-types which contain the necessary mechanisms and constraints to bridge themselves across the language barrier utilizing a symmetric type representation. Conceptually the idea can be understood as implementing the dynamic wrappers we see in CxxWrap.jl or similar libraries as zero-cost abstractions. A big motivation behind the design of Jluna was to provide an improved interface for Julia’s C-API with the extension of enabling interoperability with C++ code. As Jluna is nonetheless still built on Julia’s C-API we do still see the same limitations in performance as with other more traditional cross language tooling, i.e. the overhead of boxing and unboxing in addition to a sort of cold-start problem due to the extensive application of code-generation. One way Jluna does improve upon older tooling is that through its heavy application of template metaprogramming and conditional code generation it is able to express certain cross-language dynamics in a considerably more declarative style. The declarative nature of Jluna’s design stems in large part from the fact that as opposed to wrappers being designed as dynamic objects meant to contain a set of types, Jluna’s meta-types are more clearly defined as self-contained static type containers acting under compile-time constraints. A major benefit of this being the additional ahead of time type safety guarantees

gained both for users when defining types and for developers when extending the library.

2.3 Memory Management Strategies

Two important considerations come into play when discussing memory management in cross-language systems: the lifetime of a variable and how we handle allocation, deallocation, and access for different types across the language boundary. The implementation of languages often ties these concepts closely together, as we generally want allocation and deallocation to correspond with the expected lifetime of a variable to avoid persisting unused memory. Statically typed languages often rely more on manual control of memory, while dynamically typed languages typically implement some form of runtime garbage collection mechanism. Over time we have seen that various popular statically typed AOT compiled languages have developed approaches to limit the application of low-level memory control due to its notorious risks and often problematic usage. Examples include the Resource Acquisition is Initialization (RAII) paradigm in various C++ compilers like MSVC [44], Automatic Reference Counting (ARC) in Objective-C and Swift via the clang extension [13], the Garbage Collector in Go, and the borrow checker in Rust, which either determines implicit lifetimes at compile time or uses explicit annotations from the programmer to determine when to deallocate an object.

When examining memory management dynamics for different languages, we can observe two broad categories: languages that manage memory primarily through static compile-time constructs and languages that employ runtime implementations of heap management i.e. garbage collectors. While theoretically any statically typed AOT compiled language could have a garbage collector, historically there has been a preference for management abstractions that model low-level performant control of memory while providing enough safety and ease of use. The question then becomes how to reconcile such systems, focusing on bridging systems with different memory management strategies and types with specific mutation properties and levels of indirection. To evaluate this question, we will look at memory management strategies of interoperability systems within the context of bridging AOT languages, JIT’d languages, and AOT with JIT languages.

2.3.1 Static strategies. Central to our analysis of interoperability in the context of AOT compiled languages will be tooling built around C++ and Rust as it provides some unique insights into abstractly similar languages but with certain considerable difference in how they handle memory management. C++ and Rust are both statically typed languages which are compiled ahead of time, they both aim to provide high performance through granular control, static typing, and minimal overhead inducing abstractions. However, they employ different memory management models. C++ has traditionally relied on memory management with RAII and reference counting, while Rust uses a borrow checker to enforce memory safety at compile time. For the sake of brevity we will constrain ourselves to talking about how specifically objects; which is to say instances of records types; are managed across the language boundary. Within C++ a record type is known as a class type which encompasses non-union and union types. The lifetime of a class-type in C++ is tied to its storage duration or temporary nature, which is determined by the

context in which it is created and how its declared i.e. `static`, `new` [19]. In Rust, a record type is defined as a struct, its lifetime is determined by the borrow checker. At the core of how the borrow checker works is the concept of an ownership model. Ownership is a compile-time construct in Rust that deterministically ensures that each value has a single owner, and when the owner goes out of scope, the value is automatically dropped, placing it in an invalid state. The most straightforward approach for reconciling the semantic differences in memory management between C++ and Rust is to more or less side-step the issue through the use of thunks and *opaque type* pointers.

Thunks are adapter functions i.e. shims, which adapt arguments to fit a particular calling convention and then trampoline to the actual function. Tools such as `cxx`, `autocxx`, and the newer `crubit` [30] in large part work by generating C-ABI compatible thunks via extern "C" blocks. `cxx` handles memory layouts of types primarily through opaque types, which can most simply be understood as types which persist a pointer to some memory from a guest language within a host language. Opaque types allow us to conceptually move a type around in our host language, with a core limitation that we have no actual usage of the type aside through callbacks. `cxx` does have a extern "C++" block as a part of its C++-IDL which allows the programmer to define the a Rust-sided interface implementation which internally relies on thunks. `Autocxx` and `crubit` evolve on the idea of Rust-sided wrappers by both automating their generation process. As already discussed in section 2.2.2 `autocxx` uses `bindgen` to parse headers and then generate the C++-IDL. `Crubit` aims to avoid using a macro based IDL and instead directly generates the interfaces for the other language i.e. Rust from C++ and C++ from Rust, and the associated thunks.

Although `cxx`, `autocxx` and `crubit` generate host-language interface wrappers that internally use opaque pointers and thunks, they differ primarily in how they allocate, manage, and release the memory for the underlying guest-language objects. As these tools are primarily centered around Rust as the host-language we will focus on liveness from the perspective of Rust. An important aspect to the safe usage of objects across language boundaries is that their underlying memory stays in a valid state until both languages no longer use the object. In theory the most trivial way to avoid premature deletion of objects is by having its ownership exclusively managed within a single language. A language taking ownership of objects it uses in a cross-language setting poses certain challenges. Native Rust objects virtually all change ownership via *destructive moves*. A destructive move is one which leaves the *moved-from* object in an inaccessible state. The underlying implementation of destructive moves is more or less just a `memcpy` operation. While the approach of destructive moves works well in conjunction with the compile-time borrow-checker model it can't be trivially applied to Rust-sided memory which holds C++ objects. The subset of C++ types which do preserve their semantics within the context of destructive moves, i.e. `memcpy` moves, are called *trivially relocatable* (TR) types. Relocatability in essence aims to replicate Rust's destructive moves. Traditional moves in C++ via move constructors occur *non-destructively*, meaning the *moved-from* state must have its destructor in a valid state. Trivially relocatable types are ones which are movable and provide no user-defined destructors [12]. As of mid 2025 most C++ types which are TR can be identified as by

either being trivial, such as scalar types, and ones which are marked with the `trivial_abi` clang attribute [15], [21]³. As opposed to `cxx` and `autocxx`, `crubit` aims to represent the memory layout of TR C++ structs within Rust by mirroring the underlying layout [20]. Mirroring the C++ layout allows `crubit` to construct the TR C++ objects into Rust-allocated memory. Objects which are not trivially relocatable, which is to say objects that cannot be moved with a `memcpy` and a `destroy` are a harder problem to address. Boxing via the aforementioned opaque pointers and interface wrappers avoids this issue, but comes with undesirable performance overhead. A common example brought up of non-TR objects are self-referential classes due to the fact that when being destructively relocated the internal pointers hold old memory locations; something which C++ can manage with custom move-constructors but Rust can't. Rust introduced the `Pin<T>` wrapper type to address the issue of types which are address-sensitive which need to be pinned in memory, such as self-referential types. *Pinning* a location in memory provides the guarantee that the pinned value will remain located in the same place in memory from its creation till the time its dropped [62]. An early application of pinning to interoperability with C++ was with the `moveit` crate [72] which used `Pin` to construct C++ objects into immobile Rust memory either on the stack or heap with `Pin<Box<T>>`. The same idea of using `Pin` for non-TR memory is applied in `crubit`'s design. To avoid objects having their destructors called from the C++ sided end `crubit` typically generates cross-language thunks as methods which pass raw pointers thus avoiding either scope based or references count based destructor calls.

2.3.2 Dynamic strategies. What we saw in section 2.2.2 is that for similar languages aimed at dynamism have their interoperability typically enabled through a low-level API which allows native code to communicate with higher level constructs. How objects are represented and how their lifetimes are managed once again becomes a focal point when assessing memory management. Memory management and garbage collection algorithms are typically automatic runtime processes integrated into the runtime of a particular language. JIT compiled languages such as Julia or Java, often have their memory management models defined by some allocation heuristics and collection strategies such as mark-and-sweep. The dynamics of when collection is run is also influenced importantly by memory pressure on the system and the generation classification of objects i.e. how many collection phases an object has survived [16], [23]. Because garbage collection is managed automatically and flexibly by each language's runtime, cross-language memory persistence is typically achieved by explicitly instructing the guest language's GC-via its API-to keep referenced objects alive for as long as they are needed across the language boundary. Due to the semantic gap that exists with the underlying memory layout of object models within dynamic languages its quite rare to see any attempts holding guest-language objects directly within the host-language. Wrapper types usually are host-allocated objects which hold a reference to the foreign object and interface via the languages embedding API such as the JNI for Java, Julia's C-API or Python's C-API [54].

³Depending on if the proposal described in P2786R11 [42] is integrated into C++26 we could also see the type trait `std::is_trivially_relocatable` be introduced to more easily detect trivially relocatable types.

2.3.3 Hybrid strategies. Hybrid approaches to memory management are employed in tooling which aims to bridge languages which have fundamental gap in their memory semantics i.e. managed vs unmanaged languages. It's an aim amongst most interoperability tooling to represent types idiomatically and with low-overhead. As representing objects from managed languages within unmanaged languages is considerably more non-trivial than the other way around. As such its common to see hybrid tooling taking the shape of the dynamic language handling static types. Pybind11 is a good example of the more traditional established designs. Within pybind11 the problem of object management is solved largely implicitly by interfacing with the guest language through the embedding API. Objects are typically heap allocated by default and then wrapped in a `unique_ptr`. The C++ sided `unique_ptr` object is then wrapped in a python-sided managed object which the GC can keep track of. Once the python-sided object is garbage collected, the default behavior is to then destroy the underlying C++ object. Embedding API's, as opposed to say regular binding generator tooling, are typically designed with the purpose of idiomatically extending dynamic languages with low-level code with a heavy focus on exposing different runtime internals. The design paradigms of embedding API's are therefore quite effective at handling the memory management of objects from guest languages, provided said guest languages can generate the necessary bindings to the embedding API.

A more recent approach mentioned when discussing tool architectures was the method of embedding the static language more directly within the dynamic runtime. A nice example of this is `cpy`, which as a reminder embeds C++ within Python through the use of the `cling` interpreter. Abstractly speaking this isn't too dissimilar from `pybind`, both `cpy` and `pybind11` use Python's embedding API to expose the foreign language's API. What differentiates `cpy` from `pybind11` is that it leverages the introspective properties of the `cling` interpreter to generate the necessary bindings at runtime. In a sense `cpy` replaces the manual work of declaring the `cpp`-sided bindings with the automated introspective parsing which `cling` already implements. Despite the architectural differences `cpy` and `pybind11` ultimately end up with similar memory management properties of Python managed objects which hold references to heap allocated C++ objects. A critical takeaway from this design is that only objects which are initialized from the dynamic side, and in turn owned by the embedding API, are able to be implicitly managed. All other objects, i.e. those which are not initialized from the dynamic side, have to be explicitly managed by the programmer.

Another case to consider is how dynamic types are typically represented within static languages. One approach taken by two popular tools, namely `pybind11` and `Jluna`, is to embed the dynamic language's runtime within the static language. In the case of embedding the runtime within the static language both `pybind11` and `Jluna` have implicit proxy layers that handle boxing and unboxing. We can understand these proxy layers as essentially minimal replicas of how the dynamic language runtime interacts with the embedding API. This is to say that they generally aim to roughly emulate the semantics of the dynamic language, and in kind leverage the underlying runtime i.e. the embedded interpreter for memory management. Naturally the embedded interpreters typically also have

the ability to execute inline dynamic source code, which depending on the implementation is able to capture any scope of proxied variables as they themselves are also tied to the runtime through the embedding API.

A final approach worth discussing is specifically the symmetric type representation of `Jluna`. `Jluna` distinguishes itself from other hybrid models by its ability to completely avoid wrappers *for the programmer*. Within most interoperability tools wrappers define types which are used to represent the foreign type within the host language. Another interpretation we can have of a wrapper is that the host type which is implicitly defined by the wrappers constraints, is the actual idiomatically pure type within the host language. Wrapper constraints, broadly speaking, encapsulates a set of either static or dynamic assertions which define the semantics of how we interact with the foreign type. The most important of these constraints for record wrappers is typically the field typings, as we need the marshalling mechanisms to have a valid input type in the host which can be mapped to the target. The "front-end" representation we get from wrappers is what's then usually supposed to mirror an idiomatic type within the host language, but an important aspect is that the actual back-end semantics of the type more often than not do not conform to the host languages pure idioms. If we take for example a C++ struct from `pybind11` or `cpy`, on the python side we do get an object which has the same fields as the C++ struct, and even behaves for the most part like a python object, but the actual semantics of the type are driven by the host's embedding API as opposed to the actual host language runtime. When we talk about a purely idiomatic type we mean a type which is defined purely within host language semantics, i.e. a regular host language type. Here we have `Jluna`'s symmetric type representation come into play. As opposed to being used to persistently contain a foreign type, `Jluna` uses the notion of a wrapper as a blueprint to construct a purely idiomatic type which mirrors the foreign type. Similarly to `pybind` we describe a type to our interoperability layer, but instead of generating a wrapper which contains the foreign type and manages it, `Jluna` generates wrappers which act as specialized or type-specific translation proxies. These translation proxies are then used in conjunction with wrapper lambdas to create an invisible middle interoperability layer which yields purely idiomatic types within both the host and target language.

2.4 Type Interpretation Strategies

When we talk about type interpretation strategies we are referring to how interoperability tooling is able to parse the syntax of a type and then derive a semantic understanding from which it can form some internal representation. We've touched on some of the design considerations for how we parse types when discussing design strategies in section 2.2.1. A theme which might have already become apparent from previous mentions is that type parsing in the context of interoperability often naturally mirrors how native types are interpreted. Parallels between type parsing in interoperability tools and type interpretation in host languages offer insight into key design considerations.

Within traditional models of compilation we generally create a semantic representation of a type by first delineating the source code into units commonly called tokens through a lexer, and then

from this stream of tokens create a structural or syntactic representation of our code. A common organization of the syntactic representation is a hierarchical AST. During this *front-end* analysis we typically also perform semantic analysis, one part of which is enforcing type constraints on data. Type checking in the front-end is also generally labelled as “static” type checking. In the context of interoperability tooling we are generally exclusively interested in the semantic representation of types as opposed to the entire language. Nonetheless, due to the overlapping aim of deriving a semantic-preserving manipulatable representation of a type, compiler based parsing has had considerable influence in the design of interoperability tooling.

One way to understand traditional compiler front-ends on a high-level is that they create a view of the source code and then generate a representation of this view which can be manipulated, such as through optimizations, in a semantic-preserving way. In the case of dynamic languages the idea of viewing code often comes in the form of runtime introspection, which was touched on briefly in section 2.2.1. *Reflection* builds on the idea of introspection by also enabling self-modification of code at runtime. An important distinction of reflection vs the transformations which occur in a compiler front-end is that reflection is not necessarily semantic-preserving across the whole program. Introspection and reflection are often significantly more variable in the schema they use to represent types but due to their increasing application and development have offered another idiom from which to derive type representations.

2.4.1 Static interpreting. Two common approaches that fall into the vein of compiler, or statically-based, interpreting is either emulating compilers, i.e. building a mini compiler front-end aimed at lexing and parsing specifically headers or using a compiler’s AST parser and then transforming this into a domain specific representation. We can see the former approach in tools like `cxx` and by extension `autocxx` which use the `syn crate` [65] for lexing Rust tokens and then feeds this into a semi-custom parser which handles constructing an AST for the rust-sided type representation which can be used for validation and code generation. A generally harder task is transforming a full language AST into a domain specific representation. As mentioned in section 2.2.2 the `bindgen` tool uses the `libclang` library which provides a small and stable API for some of `clang` front-end internals to parse C/C++ headers. Using `libclang` comes with various benefits and drawbacks, a primary one described by the `crubit` team was that while `libclang` provides stability it also limits flexibility. Due to the limited flexibility provided by `libclang`, amongst other reasons, we have also seen instances that opted for a tighter integration with the `clang` front-end, such as `crubit` or the `Swift` compiler which both import features more directly from the `clang` front-end source code for parsing C++ headers [75]. `Swift` does also appear to use `libclang` in its compiler though it doesn’t seem to be the primary tool for parsing C++ headers.

2.4.2 Dynamic interpreting. Dynamic type interpretation is naturally most common in languages which offer the best facilities for introspection and reflection. Here we can see introspection and reflection being applied in various respects. Tools such as `Jluna` use the metaprogramming features provided by `Jluna` to implement a meta-type which represents a generic container for records, which is then used to construct the actual type representation of

the mirrored foreign type with the metaprogramming tooling. We also commonly see introspection and reflection applied in interop between static and dynamic languages. Since it’s common to use low-level embedding APIs to interoperate with high-level languages like `Python` or `Julia`, there’s a strong incentive to exploit the linguistic overlap between these APIs and the static guest language. This enables the creation of translation mechanisms that don’t depend on external tools (e.g., header parsers). As a result, developers often rely heavily on metaprogramming to manually specify types in a way that emulates dynamic introspection. These specifications are then transpiled or transformed into API-compliant code that can interface directly with the high-level language.

3 IMPLEMENTATION AND SYSTEM DESIGN

So to begin this exploration of the practical side language interoperability I had to first think of a piece of software I could use to test this idea. Before doing so I thought it would be important to further refine my idea beyond just wanting to test out my ‘lego idea’. As said before the concept of rewriting code is by no means new, I don’t believe it would be absurd by any means to suggest that most programmers at one point or another, while writing some piece of code, or even after the fact, recognized ways in which their solution can be improved. Often improvements or iterations on the original piece of code just naturally have to occur to fit the evolving goal for a given piece of software. So it would make sense that there should be some type of reason as to why we want to rewrite a specific piece of code, in our case performance would be a pretty natural choice. While most programming languages of course aim to be performant, respective to their application, due to differences in their design and execution methods they naturally exhibit differences.

Ok, so we want to choose a piece of software, preferably one with a performance-critical component, which we want to rewrite. Looking back at my previous projects one specifically came to mind, my implementation of a simple `Ray Tracer` in the `Julia` programming languages based on Peter Shirley’s wonderful guide and C++ implementation [47][48]. My initial goal with this project was to learn `Julia` as I had heard and seen that if written in a very idiomatic manner it can exhibit performance characteristics close to that of `C` while offering the dynamically-typed high-level simplicity of `Python`. Naturally, when benchmarking my final implementation my results were less than desirable. To elaborate on why my program exhibited these performance characteristics it makes sense to briefly go into the core of how `Julia`’s code is run. There is a very nice quote on the `Julia` forums which I believe effectively conveys this.

in short, it’s a reasonably good static compiler but a terrible JIT

The first response some people might have to this is confusion. `Julia` is a JIT-compiled language so how is its JIT compiler terrible. Well, this comes down in large part to how its JIT compilation works, the core of this is how it performs online partial evaluation (OPE) w.r.t. its type system. OPE w.r.t the type system in this instance refers to the process of, at runtime (online), `Julia` specializes (partially evaluates) methods for which it can infer the types of every local variable from the types of the arguments or any other

constants; in this instance, a method is said to be ‘type stable’. Specializing type stable methods simply means it compiles methods for the set of concrete types it can infer. Practically speaking this mechanism can also be seen as a form of monomorphizing a set of functions for which the JIT compiler can determine the concrete types. The reason then why Julia’s JIT compiler could be called ‘terrible’ would be that it, currently at least, does not implement profile-guided optimization (PGO). The lack of PGO and its reliance on type stable code for speed means that there is a correlation between idiomatically written Julia and its performance. With this understanding, it might now be apparent why my Ray-Tracer performed so poorly, as I was not fully aware of this strong dependence and I simply wrote the code how I would write something in any other dynamically typed high-level language.

We’ve now established that we have a piece of performance-critical software written in a very non-idiomatic and by extension inefficient manner. Furthermore, we’ve seen that its execution method is not only different from ahead-of-time (AOT) compiled languages but also from other JIT-compiled languages by its unique approach to generating machine code. The conventional route, and quite frankly recommended route, one should go here would be to simply rewrite everything with Julia’s idiomatic principles in mind, though this would lead to a rather short exploration, so to question the alternative, what if we rewrote the most performance-critical segment of our ray tracer in languages which have less of a reliance on a highly idiomatic programming style.

In the following I’ll briefly start by giving an overview of the ray tracer itself and then how I approached integrating the two different languages Python and C++ into this system. I’ll then also motivate or give some insights into the reasoning behind why I chose the binding generators that I did, and some of their internal architecture and how it relates to some of the things discussed previously. Once all this is established I will elaborate on the issue which I encountered as a byproduct of the design of this raytracer when working with the C++ rewrite. From this I will go into the nature of the problem and by extension the constraints that came with the design of the C++ ↔ Python bridge Jlua. With this information I will explain how I came to my approach for a solution and some of the pitfalls I had in its design. Finally I will do some benchmarking of my solution and compare and contrast this with the approach that PythonCall employs. Ending things on a discussion about the limitations and possible future directions.

3.1 System Overview and Setup

As one of the objectives of this paper is not only to research the theoretical underpinnings of interoperability but also assess their viability in practice we can move on to describing how a lot of the topics previously discussed have relevance in practice. In the preamble the decision was made to use a ray-tracer implementation written in Julia as the main subject for this exploration. Before delving into the implementation of the actual cross language aspects it would make sense to give some motivation as to why a ray-tracer serves as a good subject. The decision; despite being initially somewhat ad-hoc; does present some potentially insightful attributes. Ray-tracers, especially more developed legacy implementations, are somewhat well-known for being complex pieces of software,

not only due to their foundation in mathematical modelling but also their interconnectedness of various different systems to enable the actual image generation. The complexity of even somewhat minimal ray-tracer implementations; as in the case of the one I’m using; nicely presents a situation analogous enough to more complex systems giving potentially some broader insights into the effects of interop as opposed to some domain specific minimal example. Performance is another key aspect of ray-tracers, while they had their roots in drawing simple images nowadays ray-tracers or more broadly speaking physically based rendering (PBR) frameworks are used in real time in video-games, in turn creating a massive demand for high performance PBR systems, while the ray-tracer used in this paper only implements a very minimal subset it still relates directly to a domain in which performance is critical, again serving as an indicator of the viability of the applied interop techniques in performance demanding scenarios. Finally, despite the aforementioned complexity of ray-tracers they do tend to be very modular systems in some respects, this modularity should allow for an easier time separating predominantly logically distinct components and thus make the rewriting process easier. A high-level overview of the ray-tracer can be seen in Figure 1.

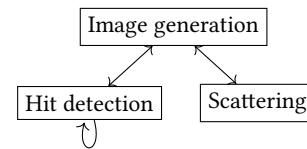


Figure 1: Ray-Tracer high level control flow

3.1.1 Implementation approach. Now that we’ve established our motivation for testing and have an implementation to work with, the next step is figuring out how to test it in practice. To isolate the most performance-impacting component, we profile the application, analyzing both runtime and memory usage before making informed judgments based on this data. Regarding language choices for rewriting, we are constrained to those with embedding APIs for Julia, but we still have several options. It makes sense to choose two languages with fundamentally different properties—Python and C++—as they provide a broad spectrum of insights applicable to similar domains. Both languages have well-established interface libraries, making them practical choices for evaluation. To test the effectiveness of the component, we use benchmarking tools to analyze its performance in isolation, along with Julia-sided tools to assess the combined performance of the component, interface code, and any overhead introduced.

3.1.2 Choice of Binding Generators. Writing the components alone isn’t enough if we can’t integrate them into the larger system, so the next step was selecting an embedding API to attach our hit functions to the ray tracer implementation. For Python, the choice was straightforward—Python/JuliaCall [52], which builds on an earlier implementation and supports a wider range of conversions between the two languages. C++, however, presented a more interesting challenge. Unlike Python, which has trivial reflection mechanisms making object conversion straightforward, C++ requires

more deliberate choices. The two main options were CxxWrap [36], which has a strong feature set but lacks user-friendliness, and Jluna [18], a newer and more user-friendly alternative that is missing some mapping mechanisms. I chose Jluna because my focus was on minimizing additional code and effort while creating usable components in another language. Jluna, leveraging modern and extensible C++ techniques, proved to be more future-proof and developer-friendly. This ties into a broader consideration: when rewriting a component in another language, if the results are positive, you ideally want minimal refactoring to integrate it into the larger system. This means choosing an API that (1) requires the least additional code and (2) minimizes the code needed to attach the component. One key takeaway from this process and something which might already be quite evident given the discussion in 2 is that working between Python and Julia is already posing significantly fewer challenges when searching for a binding generator due to the added feature set we get from both languages being designed with dynamism; and by extension runtime reflection; in mind.

3.1.3 Implementation setup. The implementation setup to test the component rewritten in Python and C++ can be seen visually in 2. The code for setting up the scene, which includes parsing the ‘.obj’ files, that describe the vertices which bound all faces in scene, loading the faces into a bounding volume hierarchy (BVH), and managing the actual image generation by drawing the pixels is all uniformly done in Julia. The isolated component, namely the tree search and hit computation for a given ray cast are what is rewritten in C++ and Python, and then replaced in the Julia implementation depending from which language we initiate the ray-tracer. In both instances we redefine the hit functions such that the Julia code is redirected to the language specific variants which act as proxies to the actual host language code.

3.2 Challenges and Design Decisions

3.2.1 Problem Statement. Now while in theory the implementation here was straightforward a major issue I encountered was related to the constraints placed in the binding generator between Julia and C++. To explain the cause of this issue it makes sense to first go into what our implementation actually needs to do, or more specifically what we are bridging in our case. As the hit detection functions; the functions which made up the BVH traversal; served as the nicest component to isolate due to their aforementioned logical separation from other aspects I chose them to serve as the bridge. Now broadly speaking the 3 main things needed when we are trying to do hit detection is our BVH which spatially structures the objects in our scene, the actual ray for which we are trying to compute a hit, and finally a hit record which stores the actual information from the hit (e.g. material, normal vector to the surface, etc.). Assessing the complexity of these types the simplest one is the ray at is just stores an object and direction, next we have the hit record; which despite being a struct; does have polymorphic attributes, namely the material of the object which the ray hit, finally we have the BVH tree which as implied in the name is a tree structure which adds another layer of complexity as we now have recursive polymorphic attributes with the left and right child being BVH nodes which subtypes that define the underlying type of the shape represent (e.g. Triangle, Sphere, etc.). Visually we can represent this as follows.

In both of these instance the type of polymorphism we are talking about is subtyping polymorphism, while both C++ and Julia; abstractly speaking; implement similar conventions in terms of enabling subtyping polymorphism, namely some abstract type and a means of specifying a type hierarchy, there are considerable differences when it comes to the practical case of interop. A very distinction between the high level notions of interop and the low level practicalities is how types are actually represented, as we are still fundamentally working through the C-ABI or more accurately through an abstraction (Jluna) over the Julia C-API before we do anything we need to actually parse the underlying representation of Julia types. Pleasantly enough this process is elegantly implemented within Jluna for all types up to structs. The manner in which Jluna does this is largely by using generics to recursively deconstruct the low level representation of a Julia type. To ensure constraints on the implementation, by which I mean rejecting types which it cannot parse for some C++ analogue the implementation utilizes constraints (the implementation being concepts in this case) on its generic deconstruction methods which ensure certain invariants hold that constrain the set of bridgeable types to only those specified by the programmer. One of these constraints was a static assertion which prevent bridging abstract types, in other words types which exhibited subtyping polymorphism, which brings us back to the issue I ran into. The question then becomes how does one solve this problem. To contrast this with the case of Python and Julia; somewhat expectedly; the story was a lot easier. JuliaCall, which is the library I chose for bridging between these two languages simply wraps Julia types with python objects that idiomatically mirror their Julia counterparts, which in turn means that we retain all the means to easily replicate the polymorphic behavior in Python. Trivially a straightforward solution to the C++ and Julia interop. would have been to simply rewrite the ray tracer such that the BVH and hit record represent everything in a non polymorphic format, though there a few reasons I was curious about trying to bridge these types.

- (1) *Challenging Common Notions:* It’s a somewhat common notion when you look at the interoperability goals for bigger companies to simply constrain the set of bridgeable behaviors to minimal things. I was curious to challenge this notion; with the increasingly expansive type systems even mainstream languages implement, sooner or later, a hyper restrictive approach to cross-language systems will substantially limit libraries which apply more complex aspects of their type system in library design.
- (2) *Flexibility Discrepancy:* Furthermore, I wanted some clarity on to what extent the general discrepancy in flexibility between dynamic languages and statically compiled languages in the context of cross-language systems spoke to some basic nature of type systems and execution models or was more a function of certain implementation philosophies or limited applications of a language’s features.
- (3) *Viability of Loosening Constraints:* I was curious to test the viability of loosening the constraints both in terms of implementation complexity of the system and the resulting performance effects that come from bridging these types.

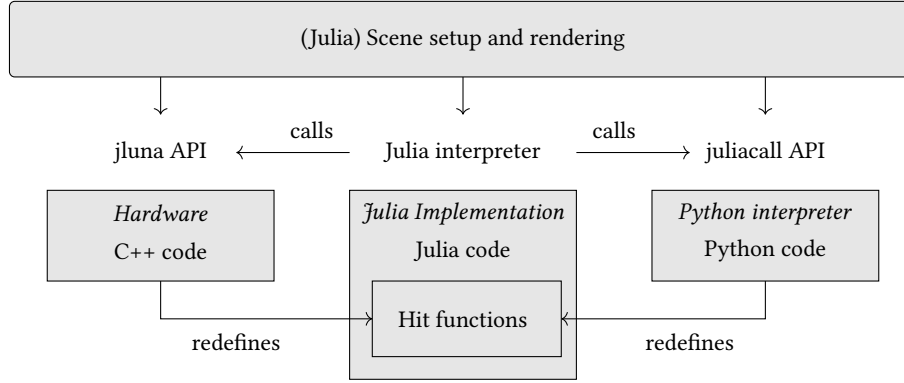


Figure 2: Cross-language implementation of the ray-tracer

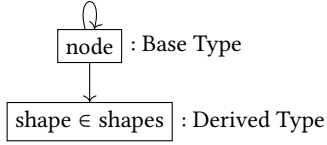


Figure 3: BVH Type Structure

- (4) *Feasibility Exploration*: I was generally curious about the feasibility as a whole, given that it's a lot rarer to see complex type bridging for cross-language systems involving statically typed languages, especially ones where the statically typed language is in focus (as opposed to relying heavily on the dynamic language). I felt this would potentially provide some interesting insights.

With the broad idea in mind of what we want to achieve now and some motivation as to why it makes sense to establish a more concrete approach. Before we assess the details for how to extend the Jluna library it makes sense to first assess in detail how the library works currently. As was discussed throughout 2 if we think of C++ as our host language and Julia as the target then boxing would be the operation of representing a C++ type in Julia, and unboxing the reverse. In Jluna the boxing and unboxing have multiple steps, we can give a high level overview of it as follows. With a general breakdown of what each step entails.

- (1) *Type specification*: For the type specification of object types (i.e. classes and structs) the programmer defines a set of getter and setter functions in addition to some other type attributes which are then encoded in a meta-type akin to meta-types generated via dynamic reflection mechanisms in dynamic languages.
- (2) *Type implementation*: Once this type is defined the programmer calls an implementation method which utilizes the meta-type to generate an idiomatically mirrored Julia type using its metaprogramming features which creates the corresponding type within the Julia runtime.

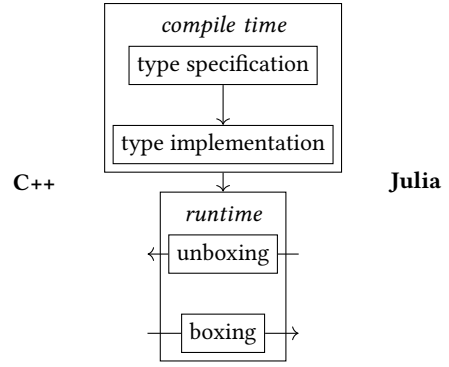


Figure 4: High-level overview of Jluna application

- (3) *Unboxing*: Unboxing is then applied to all arguments on methods which are exposed as cross-language via an implicit unboxing proxy which is generated through the specification of the function type signature in the Jluna framework.
- (4) *Boxing*: Conversely boxing is applied to all return values of the function which are then passed back to the host language, again as an implicit wrapper around the bridged function.

A first rather obvious question would be how do we implement our extension without having to massively rewrite the existing library. As mentioned before the library relies heavily on utilizing generics, more specifically template metaprogramming, which is an entirely compile time construct in C++. The focus on compile-time metaprogramming meant the type specification was naturally reliant on compile-time information, which in turn meant that the type specification was largely static. Given this emphasis on static type specification I wanted to see if it was possible to integrated a static polymorphic type specification into the existing framework; as opposed to a more runtime based process. Despite the somewhat antithetical nature of the two concepts; static compile-time type specification of a runtime polymorphic type; constrained to the paradigm in which the framework was designed, I was curious to

see if this was a feasible extension. To then approach this in a structured manner I first had to understand abstractly the the different attributes of the BVH tree which I needed to encode, and from this gain an understanding what precisely needed to be extended. If we do a high level breakdown of how you might describe the meta attributes of the BVH tree we can decompose it as follows.

Attribute	Type	Description
Abstract base class	BVHNode	An abstract base class which represents the BVH tree
Abstracted attributes	BVHNode	The left and right child nodes which represent either inner tree nodes or leaves
Inheriting classes	shapes	The shapes which exist on the leaves of the BVH tree and all exist as derived types from the BVHNode class

Table 2: BVH Type Attributes

With this in mind we can make few key observations already. As stated before the library was designed to fully disallow abstract types, hence this would likely form the first step to our extension. Furthermore as this library was not designed with abstract types in mind it naturally did not implement a way to accomodate such types, or attributes of such types, in the type implementation (Julia instantiation), boxing or unboxing. From this there are a few interesting questions which arise as to how we proceed.

- (1) *How do we encode the abstract base class?* Given that the library was designed to fully disallow abstract types, the type specification needs to implement a mechanism to express subtyping relationships.
- (2) *What are the effects of this constraint on the implementation of the library as a whole?* More broadly this is a question of extensibility in library design often appears. One interesting aspect of Jluna specifically is its heavy usage of compile-time metaprogramming and highly generic type encodings as opposed to for example concrete wrappers or more traditional parsing approaches as a means of generating bindings. A natural question which then arises is how does the extensive application of these paradigms effect the ease of extensibility given the constraints placed on the initial implementation.
- (3) *Unboxing - How do we parse the abstract Julia type in C++?* Naturally as the nature of our problem is bidirectional we need to reconcile how the low level BVH tree representation in Julia is parsed in C++.
- (4) *Boxing - How do we represent parse the abstract C++ type to create the corresponding Julia type?* Similarly we need to ensure that the abstract C++ type is not only implemented in Julia but that we also have a means of moving the C++ type to this corresponding implementation.
- (5) *Constraints - How do we loosen constraints while retaining safety?* Finally we need to make sure that we can not only represent arbitrary BVH trees but also that we establish a means of ensuring that the constraints placed on the type specification are loosened in a way that does not compromise the safety of the system. Which is to say that we should define a new set of constraints for what constitutes a valid type polymorphic type specification.

3.2.2 Metaprogramming and Partial Evaluation. Before getting into the actual code which implements this extension there are two points I feel are important to establish in more detail before we move on. I've mentioned a few times by now the use of compile-time metaprogramming in the Jluna library, and while I've given a high level overview of what this means I want to dedicate some time to explaining some of the aspects of this design paradigm in the original implemenation and furthermore how these aspects were applied to address some of the questions which were raised in the process of developing the extension. We can start by giving a quick overview of templates in C++ and how they are used in the Jluna library.

```
libs/jluna/.src/usertype.inl
```

```
1  template<typename T>
2  bool Usertype<T>::is_implemented() {
3      return _implemented;
4  }
```

Listing 6: Jluna implemented query function

In 6 we see the most basic application of a template, and one which should be familiar to most programmers who have used generics before. In this instance for the set of specified types $T \in \mathbf{T}$ the function will be instantiated for each type in the set. A few observations to make here is that the type T in this isolated instance is not bounded, which is to say that; ignoring `Usertype`; this function could be instantiated or specialized with any arbitrary single type. To allow for some level of restriction in the types which are permitted to specialize a template C++ implemented its version of constrained generics called concepts. Concepts define compile-time predicates which express a requirement on a template argument. We can see a basic application of this here.

```
libs/jluna/include/usertype.hpp
```

```
1  template<is<double> T>
2  unsafe::Value* box(T value) {
3      return jl_box_float64((double) value);
4  }
```

Listing 7: Jluna concept function

In this case the concept is used to call the most appropriate variant of the box function for the double type. What this enables is static method dispatch, in other words it uses constrained parametric polymorphism to emulate the behavior of dynamic dispatch as the constraints define a subtyping relationship. Naturally the language extends this idea to multiple type arguments which each can have their own set of constraints. A question which might arise from tseeing this language definition is what if you want to define a template with a set of types of arbitrary length. Generics which accept an arbitrary set of types are known as variadic generics, or in the context of C++ variadic templates or template prameter packs. Furthermore the concepts can also be applied to these variadic templates. Once again we can see an example of this being applied in the Jluna library.


```
libs/jluna/include/proxy.hpp
```

```
1 template<is_boxable... Args_t>
2 Proxy safe_call(Args_t&&...);
```

Listing 8: Jluna variadic concept function

On a high level all that's happening in 8 is that we are checking the predicate that for each member of `Args_t`; which represents an arbitrary length set of function arguments; the type of this argument is boxable and hence transferrable across the language boundary. In this snippet we see two new pieces of notation introduced, namely `'...'` which is used to denote a variadic generic, and `'&&'` which in this represents an *rvalue reference* which is a reference that binds to an rvalue (temporary value) which is about to be destroyed enabling move semantics for all arguments passed to the function.

A final relevant construct worth mentioning is the template metafunction, a basic example which can be seen in listing 9.

We can see a basic example of a particularly relevant construct of template metaprogramming which enables the above mentioned below. There are various interpretations of this construct depending on what it's applied to, in the most direct case it's simply a generic struct which exposes its template arguments as statically bound attributes. An analogous variant of this idea of generic objects can be seen in the implementation of traits whose attributes are used to define shared behavior for some set of implemented types akin to instantiating a generic. Traits or interfaces in most languages can then be described as being value level abstractions in that they give us an abstract or shared view of what behavior some set of types might have. Moving to a higher level of abstraction we then come to the idea of type traits, which are traits that; as opposed to describing shared behavior; they describe a type itself. In the context of C++ we can see this being applied heavily in conjunction with constraints, as if we have a means of expressing certain properties of types it's not too far out there to then use said properties to define compile-time constraint predicates. Stated differently constraints in the context of C++ are simply type traits which utilize partial specialization to implement compile-time control flow which in turn enables compile-time predicates on types. The more broad terminology of this idea is a template metafunction, which is intended to express a function which operates in terms of template arguments, or just types, as opposed to values and 'returns' a type or computation applied to this type. For completeness we should also address the fact that the example 9 has a non-type parameter. C++ allows for constant value-level parameters to be passed to templates, which can be used to define compile-time constants which should be applied in the template instantiation process. A more formal way of viewing this is that template metafunctions can be defined as dependent types.

While constrained generics are a somewhat standard feature in many different programming languages an interesting observation to make; even with just these basic constructs; is that we in essence have two different programs. The first being the program consisting of the source code the programmer interacts with, the second being the instantiated program which is generated at compile time given

```
src/cpp/example/mfuncex.cpp
```

```
1 template <typename Object_t, const char* name>
2 struct ObjectMeta {
3     using obj_t = const Object_t;
4     static constexpr const char* Name = name;
5 };
```

Listing 9: Template metafunction function

the statically defined type specifications. The code generation process by itself is a natural byproduct of how instantiating generics works, what's interesting to consider here is the extent to which we can use maximally utilize this code generation process when designing libraries. An important notion here is the idea of metaprogramming, which is the idea that, as opposed to writing programs directly, as we might in a more traditional sense, we write a program which generates a program; as the name implies working on a 'meta' or simply higher-level of control and abstraction. To be more specific we can introduce the concept of partial computation also known as partial evaluation. Partial computation, first introduced by Futamura [27], states that given a function f processing some known data k , and unknown data u we can generate an intermediate program f_k which is the result of computing everything in f with k and then computing $f_k(u)$ we can get a final result. Visually we can represent this as follows.

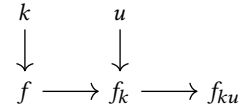


Figure 5: Partial Evaluation Process

Expressing this in more rigorous terms we can define a partial evaluator p_e as a function which maps a program p and some collection of statically known information I_s to a program p' called the "residual" via specialization, with p' then mapping the dynamic information I_d i.e. runtime information to a given output O . Formally we can express this as follows

$$p_e : \langle p, I_s \rangle \rightarrow p' : I_d \rightarrow O$$

Bringing this back to metaprogramming an observation which was noted by [68] is that this act of specializing based on the static binding of types to templates, is quite analogous to the process of partial evaluation. Something which gave further ground to this idea was the fact that templates allow non-type parameters, recursion and partial specialization. As the recursion implies self-reference and the combination of non-type parameters and partial specializations can be used to express a base case, we can define various control flow structures in the template or more broadly type level, something which was first discovered by [66]. Due to the somewhat accidental nature of the discovery template metaprogramming and the notoriously cumbersome syntax of C++ templates, the idea of applying this paradigm in any extreme length has largely

been relegated to niche instances as only more recent versions of C++ have tried to really focus on expanding the feature set of templates.

Some important observations we can take away from this similarity between template metaprogramming and partial evaluation is that firstly we can view the process Jluna defines to specify and implement types as a form of partial evaluation. Specifically we can consider jluna as a binding generator-generator which is to say that jluna represents a partial evaluator which takes the type specifications and the generic logic used for bidirectional transformations and defines a specialized binding generator which is then compiled. Expressed in formal terms if we consider jluna as a function G_{L_1, L_2}^1 which maps a set of types specifications T and a set of binding generator logic C to a binding generator C_T we can express this as follows.

$$G_{L_1, L_2}^1 : \langle T, C \rangle \rightarrow C_T \quad (1)$$

If we view G_{L_1, L_2}^1 as being an interpreter for a given type specification then we can see this as representing the first futamura projection. Abstractly speaking futamura projections express different stages of ‘meta’ computation, the first projection represents what was described above, a partial evaluator which specializes a program based on some known information. The second projection then represents the notion of specializing the specializer for the interpreter. In the context of binding generators this would be the process of specializing the binding generator for a particular set of languages. Formally we could define this specializer as a function G^2 which maps a pair of languages $\langle L_1, L_2 \rangle$ to a binding generator G_{L_1, L_2}^1 as follows.

$$G^2 : \langle L_1, L_2 \rangle \rightarrow G_{L_1, L_2}^1 \quad (2)$$

In practice implementing G^2 would require us to work at a higher level of abstraction, now instead of generic logic which is meant to apply to the set of types we wish to bridge we define the abstract attributes of a binding generator itself. What this means for example is that boxing and unboxing logic would now be defined as specializers methods given statically bound language specifications by the programmer whereby the logic would create concrete variants which functioned over some set of bridgable type specifications. Currently no binding generator I am aware of implements a full second futamura projection, an example which comes is a framework like Simplified Wrapper and Interface Generator (SWIG) [26] though while this tool technically accept arbitrary target language specifications its host languages are limited to C and C++, furthermore the tool does not specialize a binding generator for the languages instead it directly generates the bindings based on the parsed C/C++ headers.

The complexity of implementing a full second futamura projection is in large part a function on the degree to which the language specifications describe the required interoperability operations. In the most trivial case the language specifications would describe most of the underlying logic in which case G^2 would only have to ensure that the specification of this logic exists in a form which can be form which can be specialized. Conversely the most extreme case would require the specializer to implicitly derive or account for the entire interop logic by figuring out what would constitute the set of valid type specifications and the corresponding logic to bridge

them. A reasonable middle ground would be to define a Domain Specific Language (DSL) which could transpile to a more complete language specification essentially serving as a proxy between the programmer and G^2 . Using a DSL as a means of abstracting away complexity has already been seen in the work of Zhang et. al. [76] which used a DSL to unify kernel programming. A notable feature of this DSL was that it is defined entirely within C++ by using template metaprogramming in addition to macro expansion. The nature of having a DSL embedded which is based on compile time constructs means that we can provide custom high level abstractions while reusing all existing language functionality and having intrinsically seamless interop with the host language.

An important thing to take away from the idea of thinking about binding generators as partial evaluators is that it we end up with a more well defined separation of concerns between different steps of the binding generation process which can offer up a more effective foundation for implementing future generators in a more uniform and preformant manner by isolating the statically known specifications from the actual implementation logic and viewing the implementation itself as a specializer over the specification as opposed to being a direct implementation of interop logic. The relevance here being that it supports the fact that any extensions made to the type specification should be done in a way that does not compromise the existing logic of the binding generator to ensure we maintain this quality of separation of concerns.

3.2.3 A Record Type Formalism. If we take this idea of viewing Jluna as an interpreter of a type specification grammar we can create a basic formalism which can help us in providing a more rigorous basis for our extension. First we establish the existing specification grammar and application rules, from which we can build a basic formal model of the relevant algorithms. From this we can expand the syntax and incorporate the new rules which would allow for bridging abstract types.

$$p_t \langle x, y \rangle = \lambda x. \lambda y. \lambda f. f \ x \ y \quad (3)$$

Expanding this to support typed pairs we introduce the notation from System F

$$p_t \langle x : \tau_x, y : \tau_y \rangle = \lambda x : \tau_x. \lambda y : \tau_y. \lambda f. f \ x \ y \quad (4)$$

Furthermore we define accessor functions for the pair as follows

$$(a_1 : p_t \rightarrow x) \leftrightarrow (\lambda p. p \ (\lambda x. \lambda y. x))$$

$$(a_2 : p_t \rightarrow y) \leftrightarrow (\lambda p. p \ (\lambda x. \lambda y. y))$$

Now with this we have the notion of a idea of a field type defined but we still need to actually define how we can represent some collection of these fields. Informally we can describe a record type as being a collection of labels and corresponding types, with the instantiation associating values with these labels [17]. One drawback with this approach is that it does not express the nuance of what record types actually represent. In a more nuanced perspective we can think of record types as being dependent n-ary product types which are indexed by a set of labels. To break this down into different parts we can first consider n-ary product types.

Definition 3.1 (n-ary Product Type). An n-ary product type, generalizes the notion of a standard product type between two types to n types. Which is to say that it expresses the sets of all possible

n-tuples of types. Formally given a set of types T we can define the n -ary product type as follows.

$$T_1 \times \dots \times T_n = \{(\tau_1, \dots, \tau_n) \mid \tau_i \in T_i, \forall i \in [1, n]\} \quad (5)$$

We can then consider a dependent type can be thought of as a type which can vary over a set of values, as we saw before with the case of non-type template parameters for C++.

Definition 3.2 (Dependent Type). Expressing this formally we can say that a dependent type is a type which is indexed by a set of values belonging to another type, formally this idea is represented as follows.

$$x : A \vdash B(x) : \text{type} \quad (6)$$

Which is to say that for some value of type A we can define a type $B(x)$.

A dependent type by itself only describes the family of types indexed by another type. A common example of a dependent type is if we consider A to be the set of natural numbers and B to be vector. The dependent type $B(x)$ then represents the family of types consisting of all vectors of length $x \in \mathcal{N}$. If we wish to express the collection of dependently typed function which assign each $x : A$ to an element of $B(x)$ we can express a dependent product type as being formed out of a type A and a dependent type $x : A \vdash B(x)$.

Definition 3.3 (Dependent product type). A dependent product type denoted $\prod_{x:A} B(x)$ is a type which represents the collection of all functions which assign each $x : A$ to an element of $B(x)$. We can represent the formation of this type using the following rule

$$\frac{\text{DEPENDENT-PRODUCT-FORMATION} \quad \Gamma, x : A \vdash B(x) : \text{type}}{\Gamma \vdash \prod_{x:A} B(x) : \text{type}}$$

For completeness we can demonstrate what application of a dependent product type looks like for a given input.

$$\frac{\text{DEPENDENT-PRODUCT-APPLICATION} \quad \Gamma \vdash f : \prod_{x:A} B(x) : \text{type} \quad \Gamma \vdash a : A}{\Gamma \vdash f(a) : B(a)}$$

This should hopefully make the idea clear that once we actually apply some input we can see quite trivially the dependent nature that is being expressed here in the return type of the function f . From this it might already be clear how we can apply this to formalize the idea of indexing into a record type. To get the last piece of the puzzle we have to describe how indexing would work for an arbitrary input. Which is to say we want to derive a λ -abstraction which contains a term of type $B(x)$ and a free variable $x : A$. We can express this as follows.

$$\frac{\text{DEPENDENT-PRODUCT-INDEXING} \quad \Gamma \vdash x : A \vdash b : B(x)}{\Gamma \vdash \lambda x. b : \prod_{x:A} B(x)}$$

Finally we can express the trivial β -reduction rule applied to the λ -abstraction.

$$(\lambda x. b) a \rightarrow_\beta b[x := a] \quad (7)$$

With this we can now express the idea of a record type formally.

Definition 3.4 (Record Type). If we now assume that we have a set of labels defined by a type $L : \mathcal{U}$ and a family of types $T : L \rightarrow \mathcal{U}$. We can define a record type as being a dependent product type indexed by the set of labels, substituting the new notation in for the created λ -abstraction we can express the constructor for a record type as follows.

$$\Lambda L, T. \lambda x^L. \lambda \tau^{T(x)} : \prod_{x:L} T(x) \quad (8)$$

To show how this lambda function arises we can express the formation rule for dependent product types using more functional but equivalent semantics.

$$\frac{\text{RECORD-INDEXING-FORMATION} \quad \Gamma \vdash L : \text{type} \quad \Gamma, x : L \vdash T_x : \text{type}}{\Gamma \vdash (x : L) \rightarrow T_x : \text{type}}$$

$$\frac{\text{RECORD-TERM-INTRODUCTION} \quad \Gamma, l : L \vdash \tau : T_l}{\Gamma \vdash (l \rightarrow \tau) : (l : L) \rightarrow T_l}$$

From this it should be trivially clear how application then acts as just an instantiation of the record indexing process. Stated differently we can now see how on the type level a record represents an abstract mapping between a label set and type set, with the instantiation then representing the actual concrete mappings between labels of this set and values of the corresponding types.

To incorporate the idea of subtyping into our description we can describe a high level correspondence between the implementations of structural subtyping and nominal subtyping to the context of injectivity and surjectivity when viewed a dependent type theoretical lens. We will only superficially describe the idea fiber as developing the full categorical semantics would be out of the scope of this work.

Definition 3.5 (Fiber of a Morphism). The fiber of a morphism or function $f : A \rightarrow B$ over a point B is the collection of elements of A that are mapped by f to this point. Informally we can denote this as.

$$\text{fiber}(f, b) = \{a : A \mid f(a) = b\}$$

We can then move on to defining the idea of injectivity and surjectivity in the context of dependent types. From which we can form a connection between our formalism of record types and the idea of nominal and structural subtyping.

Definition 3.6 (Surjective Function). A function $f : A \rightarrow B$ is said to be surjective if for all terms $b : B$ the fiber of f over b is inhabited. Here all we mean to express is that for every element in the codomain there is some element in the domain which is mapped to it. On a high level we can think of this expressing the quantifier of existence or in the set theoretic sense ‘at least one’.

$$\text{isSurjection}(f) := \prod_{b:B} \parallel \text{fiber}(f, b) \parallel \quad (9)$$

The notation $\parallel \text{fiber}(f, b) \parallel$ is the propositional truncation of the type expressing the fiber of f over b . For our purposes its sufficient to understand this as expressing the predicate that the fiber of f over b is inhabited, which is to say that there exists some element or term in the domain which is mapped to b . In essence it simply states the aforementioned quality of existence.

Surjection has a general correspondence to the more concrete idea of nominal subtyping. Specifically we can think of it expressing the idea that if there exists some abstract class A then we will always have at least one concrete class B which is a subtype of A . An important observation to make is that surjection allows for us to map multiple elements; in our case subclasses; to a single element; the superclass, a byproduct of this quality is that we do not have the property of uniqueness. We can view this analogous to the fact that in nominally typed languages unless we preform a, usually runtime, downcast an object only has those properties which are defined in its superclass.

Definition 3.7 (Injective Function). A function $f : A \rightarrow B$ is said to be injective if for all terms $b : B$ the fiber of f over b is a subsingleton. By which we mean to express that the collection of elements A which are mapped to b consists of either a single element or no elements at all. Here again on a high level we can think of this expressing the quantifier of uniqueness or in the set theoretic sense ‘at most one’.

$$\text{isInjection}(f) := \prod_{b:B} \prod_{a:A} f(a) = b \quad (10)$$

Injection then has a correspondence to the idea of structural subtyping. Structural subtyping; often seen more in dynamically typed languages; relies on implicit structural comparisons to express the idea of substitutability. The key distinction to nominal subtyping being that we preserve the uniqueness quality of objects, in practice this idea is often leveraged in allowing for more flexible usages of existing implementations. Uniqueness of object mappings means that unlike in the case of nominal subtyping where we typically have to downcast to access the full set of properties of an object, in the case of structural subtyping excess properties are preserved and at least in theory can be accessed without any downcasting as there is a unique mapping back to the original object which preserves the structure.

Of course this is somewhat of an oversimplification as type systems don’t necessarily implement these ideas in mutually exclusive ways or even in the most direct way. Nonetheless for the applications of our extension we can use these ideas to develop at least a somewhat more precise language to discuss the steps taken to extend the Jlna library. To then summarize and elaborate on how we can proceed, we have now developed a basic way we can more precisely use to speak about record types; the primary focus of our extension; furthermore we have a way of understanding how we can express subtyping in terms of the same theory we used to talk about record types, namely dependent type theory. We also more broadly have established that metaprogramming and binding generators for that matter can be understood as partial evaluators on some static specification. Using this groundwork we can build some key ideas which we can directly apply in the construction of our extension, these ideas or takeaways will be developed in the next section.

We can first look at the record type again, specifically lets deconstruct what we can consider the field type.

Definition 3.8 (Field Type). A field type is the dependent type which is indexed by the set of labels. We can interpret this as the

function we formed in the record indexing formation

$$(x : L) \rightarrow T(x) \quad (11)$$

We can see that this field type is an abstract view over the more concrete mapping we think of as labels to values of a particular type. When we instantiate this type we specialize it to some concrete mapping of labels to value. Or more accurately a compiler or interpreter would specialize this type to a concrete mapping of labels to values. If we go back to the analogy between a binding generator as an interpreter and put ourselves in the shoes of this binding generator then a field type as shown above would be considered already specialized for some set of labels and types. A question which we must ask ourselves then, what is the type of this field type?.

Lets first consider again what a type is. From a high level set theoretical sense we can think of a type as characterizing a set of elements belonging to this type. Alternatively; as demonstrated by the Curry-Howard Isomorphism; we can understand types as being constructors which encode some set of propositions and terms of this type as being proofs of these propositions. A common informally shared aspect regardless of interpretation being that types in a sense are just abstract descriptions. To then get back to the question at hand we can say that the type of the field type is abstractly speaking simply a descriptor of the field type itself. Just by observation we can create an informal description describing the properties of this type.

- (1) *Label*: We can confidently say that all fields clearly must have a label $l \in L$.
- (2) *Output*: We can also see that field types have some dependent output type $T(l)$.
- (3) *Record type*: Each field is also associated with a record T .

Informally a natural way we might then represent a constructor for this type could be as follows

`paper/code/constructorex.py`

```
1 class FieldType[Label_t: str, Out_t, Rec_t]:
2     label: type[Literal[Label_t]]
3     out_t: type[Out_t]
4     rec_t: type[Rec_t]
```

Listing 10: Field type constructor pseudocode

A few notes we can make here is that firstly as labels are generally strings we can express them as being concrete value types bounded by strings. Concrete value types bounded by some more abstract type such as strings or integers are commonly also called literals and can be understood as a form of static narrowing of a type. Secondly as we are working on an aforementioned higher level of abstraction the types of the attributes of field types are descriptions of the underlying types which is expressed via the ‘type[T]’ notation representing the type of the type.

A key issue we find with this is that while we have described a kind of recipe for how we might represent the type of a field, currently the description in 10 does not actually tell us anything about how we are supposed to use a field. So we need to expand this description, the somewhat trivial question which then appears

is what does it mean to use a field type? We can quite easily say this as being some operation to get a value from the field, and some way to set a value to the field. So it would be reasonable to think that we want to expand our description to include two functions, namely a getter and a setter. To be more rigorous we can express these functions as follows.

Definition 3.9 (Field Accessors AKA Lenses). We can describe the getter of a field as a function which takes a record of type T and returns the value of the field in the record defined by T_x . A setter conversly is a function which takes as an input a record and some value of type T_x and returns a new record with the value of the field set to this value.

$$\text{get} : T \rightarrow T_x \quad (12)$$

$$\text{set} : T \times T_x \rightarrow T \quad (13)$$

Unsurprisingly this notion of field accessors has already been extensively studied, in various contexts. A common first way lenses are described is through the idea of database transformations in terms of the view (get) and update (set) problem. Commonly this is then placed in a more genral context within functional programming to describe the aforementioned idea of just getting and setting values. The abstract idea of different forms of data accessors is generalized to the concept of *families of optics* of which lenses are the most well known part.

In our case for the case of field getters and setters its sufficient to constrain ourselves to the notion of lawful lenses.

Definition 3.10 (Lawful Lenses). Since we don't want our getters and setters to just behave arbitrarily as that would be a bit absurd we can define a set of laws which we expect our lenses to follow. Specifically we can enumerate what is defined as a lawful or well-behaved lens. There are 3 main rules which characterize a lawful lens. If we consider a source S (analogous to our record type) and a view V (analogous to our field type) we can define the following laws. One note on notation is that different literature uses different notations for getting and setting, in this case we will use the notation get and put.

$$s : S \vdash \text{put}(\text{get}(s), s) = s \quad (\text{Get-Put})$$

$$v : V, s : S \vdash \text{get}(\text{put}(v, s)) = v \quad (\text{Put-Get})$$

$$v_1, v_2 : V, s : S \vdash \text{put}(v_2, \text{put}(v_1, s)) = \text{put}(v_2, s) \quad (\text{Put-Put})$$

In plain english the rules translate to the following.

- (1) *Get-Put*: If we read a value then write this value there should be no change.
- (2) *Put-Get*: If we write a value then read a value we should get the value we wrote.
- (3) *Put-Put*: If we write a value then write another value we should only get the last value we wrote.

The first two qualities describe lenses which are *well-behaved* with the inclusion of the third rule giving us a *lawful* lens.

With this we can now expand our description of the field type to include how this type can be used. Adding to our previous example we end up with the following constructor. For clarification 'Callable[[A], B]' is equivalent to $f : A \rightarrow B$.

paper/code/constructorex2.py

```
1 class FieldType[Label_t: str, Out_t, Rec_t]:
2     label: type[Literal[Label_t]]
3     out_t: type[Out_t]
4     rec_t: type[Rec_t]
5     get_t: type[Callable[[Rec_t], Out_t]]
6     set_t: type[Callable[[Rec_t, Out_t], Rec_t]]
```

Listing 11: Expanded field type constructor pseudocode

As this pattern is broadly based on well studied and somewhat intuitive principles we see it often form the foundation for different kinds of wrapper types or type specifications which aim to find some abstract way to characterize and constrain the properties and behaviors of types. We can see this in the original implementation of Jluna in 12.

j1luna/.src/usertype.inl

```
1 template<typename T>
2 template<typename Field_t>
3 void Usertype<T>::add_property(
4     const std::string& name,
5     std::function<Field_t(T&)> box_get,
6     std::function<void(T&, Field_t)> unbox_set
7 );
```

Listing 12: Jluna add property function

In the case of Jluna the upper template parameter T is that of the abstract record type, in Jluna these are called `Usertype<T>`. As the name implies `Field_t` is the type of the field, in our case this corresponds to T_x . The function then takes 3 arguments, a name of the property i.e. the label, the getter `box_get` and the setter `unbox_set`. An important takeaway from this example is that as all information is statically known the compiler specializes all property specifications of T at compile-time.

Given that we now have a way of expressing record types, and an informal way of describing thier fields we can use the idea of lenses introduced to create a more precise descriptor of how we would express the type of a record type. The first thing we introduced when starting to talk about the formal semantics was the idea of the dependent product type, just as we applied this as a means of defining a record structure so too can we apply it to define the structure of a lens. We can deliniate the structure of our lens into 3 parts.

- (1) *Type*: The type on which the lens operates.
- (2) *Semantics*: The semantics of the lens, i.e. the getter and setter.
- (3) *Laws*: The laws which the lens must follow, i.e. the lens must be lawful.

To express this formally we can define the lens type as follows.

Definition 3.11 (Lens Structure). Given a dependent $T(x)$ indexed over a set of labels L we can define the structure of a lawful record

lens for this type as follows

$$\text{Lens}_L = \begin{cases} T : (l : L \vdash T_l) : \text{type} \\ \times (\text{get}_{T_l} : T \rightarrow T_l) \\ \times (\text{set}_{T_l} : T \times T_l \rightarrow T) \\ \times (\text{rw} : (x : T_l, \tau : T) \rightarrow \text{Id}_{T_l}(\text{get}_{T_l}(\text{set}_{T_l}(x, \tau)), x)) \\ \times (\text{wr} : (\tau : T) \rightarrow \text{Id}_T(\text{set}_{T_l}(\text{get}_{T_l}(\tau), \tau), \tau)) \\ \times (\text{ww} : (x_1 : T_l, x_2 : T_l, \tau : T) \rightarrow \text{Id}_T(\dots))^4 \end{cases} \quad (14)$$

A new construct introduced here is the idea of the identity type ‘ Id_T ’, this expresses the idea of equality between two terms of type T . Hence the inclusion of the functions ‘ wr ’, ‘ rw ’ and ‘ ww ’ act as laws over the type T . In other words we are encoding the laws that we want our lens to follow into the structure of the type over the records.

With this we have a baseline for how we can talk about the structure and dynamics of our record type from the more abstract perspective that we work with in the case of binding generators. Particular relevance in our definition of the Lens is the base type, i.e. the abstract record type. The abstract record type represents basic the language agnostic model of objects with both languages need to conform to in order to be bridged. There are various different approaches to ensuring this conformance to an abstract model that typically depend on the languages being bridged. A very basic approach if you have compatible languages is to simply generate bindings in a conformant manner as a preprocessing step via header parsing, in this instance a binding generator would usually either ensure that the generation process is implicitly conformant and expose idiomatic types, or it would generate wrappers which have the aforementioned added flexibility at the cost of idiomatic usage and depending on the implementation performance. The approach taken by Jluna is to generate idiomatic bindings where the conformance rules for the types structure are defined implicitly through the application of the type specification in the runtime generation process. Julia allows for this through its implementation of metaprogramming enabling the ability to define or specialize a set of types at runtime given an abstract description of different properties of this type.

3.2.4 Case Study: Jluna’s Internals. In the most abstract basic sense we can think of boxing and unboxing to simply express the idea of a cross language assignment. The general goal with binding generators, at least those which focus on very idiomatic language usage, is to provide enough abstractions such that working with two languages feels as if you are working with one. More concretely we can say that the goal is to mitigate, as much as is possible within certain constraints, the manual specification of bindings and type marshalling code for handling incompatibilities. Hence if we go back to our basic idea what we would then strive for would be something in the following form.

If we then compare this to the best analogue in Jluna we have this.

Here ‘Main’ represents the module scope and ‘safe_eval’ is a wrapper around the Julia C-API function which uses the runtime to evaluate a snippet of code and return anything specified in a low

```
example/assignmentgoal.cpp
```

```
1 MyType my_type_object = foreign_function();
```

Listing 13: Cross language assignment goal

```
example/jlunaproxyassign.cpp
```

```
1 RGBA cpp_rgba = Main.safe_eval("return RGBA()");
```

Listing 14: Jluna proxy assignment

level representation. As might already be obvious then from the code itself we clearly have some implicit transformation happening. It should also be clarified that naturally for this to work we need an existing mirrored type representations. As we established before Jluna works quite explicitly by delineating its framework into specification and logic, the specification being the type lens. Furthermore we know that Julia itself has a variant of lenses, or more simply speaking it has accessor methods for objects. The main question that arises here what is the implicit transformation happening? The most apparent answer is well clearly its unboxing, if we have an implicit transformation for some low level representation then the assignment must be unboxing. While this is accurate it doesn’t really address the relevant more nuanced point of what unboxing is and for that matter boxing really is.

We can think of it on an intuitive level by going back to the analogy between types of types as recipes. In this context we can think of a recipe as having 3 important parts, the ingredients, the instructions and the tools. The analogy here being that ingredients represent the abstract properties of our constrained type, the tools how we handle these ingredients, which leaves us with the instructions which explain how we use the tools in conjunction with the ingredients to get our final product, namely the desired type. Since we have the tools and ingredients, we then know that the instructions then must represent the logic of how we use these tools to construct a type. Unboxing and boxing then reduce to the question of, for a language A , given the ingredients (our type representation) and the tools (data accessors) how do we use these to construct the type in language B and vice versa.

Some important takeaways from this are that a type representation; regardless of any explicit specification; will ultimately always be a function of the tools, ingredients and instructions, i.e. the complexity of the type we can construct in the corresponding language only aligns with the specification insofar as their fundamental overlap. Secondly, there is a dependency between tools, instructions and generality of constructions. The more a tool is catered towards a specific means of construction the less dependence there is on the instructions though this comes at a cost of losing generality as the tool is inherently less flexible in its application. Finally, the instructions themselves are a function of the tools and ingredients, in the context of binding generators this means that the logic of how we construct a type is a function of the data accessors and the type representation.

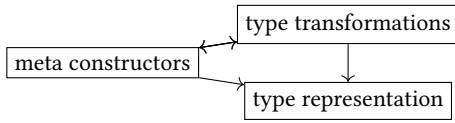


Figure 6: Basic dependencies of the binding generator

In 6, ‘meta constructors’ are just the type construction tools (e.g. lenses) and ‘type transformations’, in this context, is just boxing and unboxing. What follows from this is that, if we do want to extend a type representation; which in our case we do; we have to consider a few points.

- (1) *Ingredients:* We have to clearly define what the extended ingredients are for both languages, i.e. what are the new properties of the type representation.
- (2) *Tools:* We have to ensure that our tools are sufficient for working with the new ingredients, i.e. do we have the necessary data accessors.
- (3) *Instructions:* We have to define additional logic for how to use the tools in conjunction with the ingredients to construct the new type.

With this we have sufficient knowledge to understand how the original implementation follows from the ideas introduced. Broadly speaking we can consider two forms of types, fundamental and grouped. A record would represent a grouped type, having fields with either fundamental types or grouped types. As we can’t trivially bridge a record (as it has unbridged members) we must first bridge the fundamental types before the record. To bridge any type, there needs to exist a context in which we have access to our meta constructors, boxing and unboxing logic, and type representations. Furthermore we need some way to access this context. To approach taken by the author to approach this can be seen below

```

jluna/usertype.inl

1  template<typename T>
2  template<typename Field_t>
3  void Usertype<T>::add_property(
4    ...
5  ) {
6    _mapping.insert({Symbol(name), {
7      [get](T& t_inst) -> unsafe::Value* {
8        return jluna::box<Field_t>(get(t_inst));
9      },
10     [set](T& t_inst, unsafe::Value* val) -> void {
11       set(t_inst, jluna::unbox<Field_t>(val));
12     },
13     Type((jl_datatype_t*) jl_eval_string(
14       as_julia_type<Field_t>::type_name.c_str())
15     )
16   });
17 }

```

Listing 15: Add property function

In listing 15 we can see how the author related the type specification to a context in which it could be applied to preform bidirectional mappings of this type. The dictionary ‘_mapping’ associates the field name ‘symbol’ (wrapped by a Symbol class) with two lambdas which capture the meta constructors (getters and setters) within their context through C++ lambda capture notation ‘[captured](...args) -> ret_ty { }’, and a wrapped type representation. In this case we can see that when ‘Field_t’ is a fundamental type then the box call will be to a simple boxing method, e.g. for primitive conversions. When ‘Field_t’ is a record type i.e. it has itself more fields (assuming it has a UserType instantiation) we call its box method, which in turn will call the box methods on its field. For completeness I have an adapted pseudocode version of the box and unbox approach of the original implementation in 16.

```

jluna/usertype.inl

1  template<typename A>
2  A Usertype<A>::bridge(B* in) {
3    auto out = A();
4
5    for (auto& pair : _mapping)
6      std::get<bridge_tool>(pair.tools)(out,
7        jluna::safe_call(
8          B_get("accessor"), in,
9          (B*) pair.symbol
10        )
11      );
12
13    return out;
14 }

```

Listing 16: Bridge implementation

The bridging code in 16 is the simplified representation of box and unbox whereby we simply iterate over the fields as specified in the mapping and for each field pass of our bridging logic to the meta constructors within the map who then either bridge the type directly by calling to the fundamental bridging code or as mentioned before call to the corresponding bridge method for the record type of a particular field and so on.

A very core attribute of the implementation is that it fundamentally relies on nested records existing as fully walkable trees, in other words the bridging functions must always be supplied with concrete types such that there exists a path over an object which can be fully traced. Hence this implementation does not account for the case where we encounter an abstract type. The problematic nature then being that our implementation of the BVH tree is fundamentally based on abstraction for its representation, or more concretely as stated before we have polymorphic nodes represented by an abstract class. The aim of the extension is then first and foremost to find a way of how we can tell this process what it should do if it does encounter an abstract type.

3.3 Solution and Language Extension

Let’s start off our solution by creating a high level breakdown of the user-facing and internal flow of the library which can be seen in 7. We could tackle this problem in a linear fashion though this could

end up being somewhat problematic since we could end up defining a specification which is misaligned with the actual type mirroring process. The issue here goes back to the different dependencies we have in the binding generator as shown in 6. Intuitively it makes sense to start with the abstract type representation, ensuring that it can even be mirrored in any functional sense, as both the meta constructors to create this type and the type transformations rely on there existing some fundamental analogue for type representation in both languages.

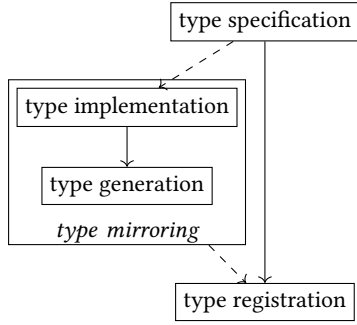


Figure 7: Stages of Jluna's type binding generation process; $\dashrightarrow$ represents user facing flow in how the user interacts with the library; $\rightarrow$ represents the internal flow of the library

3.3.1 Abstract Types & Substitutability. In our case there are two key inter-related things we wish to extend the library with. Firstly a support for abstract types, and from this a support for concrete types which implement this abstract representation. Abstract Data Type's (ADTs) in the theoretical sense simply means to express a type which encodes a contract or model for a set of behaviors and properties which a concrete variant of this type must adhere to. A basic example could be the abstract definition of a list which is commonly parameterized with a type argument representing various concrete variants such a list of ints or a list of strings, the common semantic properties then being things like the ability to add, remove or access elements. A somewhat trivial benefit of this is that unless we must know the concrete type of a list we can work with this abstract definition and hence write more generic code which can work for a variety of concrete variants. Polymorphism is a natural byproduct that comes from the idea of substitutability of ADTs, if an ADT represents an abstract contract which a set of concrete types model then we can say that any type following this contract could reasonably be substituted in place of this ADT without any change in behavior, this is the general idea of polymorphism or more specifically behavioral subtyping. A very strong condition of this idea was formalized by Liskov in her Substitution Principle.

Definition 3.12 (Liskov Substitution Principle). We denote $\phi(\tau_T)$ as a property provable about objects τ_T of type T . Then $\phi(\tau_S)$ should be true for objects τ_S of type S where S is a subtype of T denoted $S <: T$.

$$S <: T \rightarrow (\forall \tau_T : T) \vdash \phi(\tau_T) \rightarrow \phi(\tau_S)$$

As this is a quite strong condition its often relaxed in implementations of subtyping, it nonetheless serves as a good basic formalism for how to think about the idea of substitutability.

One common practical instance in which we see behavioral subtyping occur is in the case of interfaces. Interfaces within the context of programming languages refers to implementations of the aforementioned idea of an abstract type. Behavioral subtyping is then usually seen in the dynamics of interface implementation. For the case of many nominally typed object-oriented programming languages we see this take the shape of an abstract class representing the interface and an inherited concrete class forming the implementation. It should be noted that while inheritance can allow for subtyping it does not necessarily imply it, likewise the presence of subtyping does not imply inheritance. Another paradigm which has emerged often as a solution of the deficits that come with inheritance based subtyping is the idea of composition based subtyping, with a common phrase being 'favor composition over inheritance'. Composition based subtyping is similarly based on the idea of an abstract interface which dictates a set of behaviors and properties which a concrete implementation must follow, the key distinction being that a concrete implementation does not inherit from the abstract interface but rather is composed of it. In other words, for composition based subtyping the concrete implementation exist as an instantiated version of the abstract interface directly rather than an inheriting subclass. Implementations of composition based behavioral subtyping are often seen in the form of protocols, traits or interfaces, all of which are used to express the idea of a contract which a concrete implementation must follow. To clarify again the difference here being that with composition based subtyping there is no hierarchical relationship between the abstract interface and the concrete implementation, the concrete implementation is simply a realization of the abstract interface. Composition based subtyping can be understood more broadly as an implementation of parametric structural subtyping or more simply generics in which substitutability is determined by the structure of the object which exists as a specialization of the general interface.

A very important point of consideration is that since subtyping is a function of a languages type system, how do its enforcement properties influence the dynamics of subtyping. We can categorize enforcement into two main qualities, when enforcement happens, and how it happens. Since we are concerning ourselves specifically with substitutability the question becomes when do languages enforce this and how do they enforce it. When it comes to the 'when' the trivial approach would be to delineate compile-time and run-time, though as JIT compilers in a sense combine the two it makes more sense to state it in terms of information availability. We can define some hypothetical object o which is of type S , and we have a method taking an argument of type T where $S <: T$, clearly the most trivial variant of substitutability would occur in the case where $S = T$, and this is precisely what we tend to see with parametric structural subtyping, here methods are specialized such that for all concrete variants of a type T we have a corresponding concrete method accepting this variant. For statically typed languages we see this as generic methods being instantiated through means like the aforementioned monomorphism though it can also occur using

runtime based approaches. In the case of dynamically typed languages there is a little more nuance depending on implementation, the untyped case, which is to say purely structural subtyping, is defined by the semantic properties of how the argument is applied. The typed case would refer to dynamic languages which have a gradual type system, here there is considerable variation in not only the execution method (interpreted, JIT compiled) but also the extent of enforcement and application. On one end of gradual typing systems we have TypeScript which exist in large part separate from the underlying execution of JavaScript and simply acts as a layer on top of JS. On the other end we have languages like Julia which; despite having a sort of gradual typing; makes considerable use of the type annotations as a means of more effectively specializing code with the aim of minimizing runtime compilations. Nominal subtyping on the other hand tends to be a predominantly run-time process regardless of language, the main reason being that with nominal subtyping we express substitutability as a function of an explicit hierarchy as opposed to direct static equality. The hierarchy is then something which must persist and be used for us to determine a valid substitution as opposed to it being trivially and apparent. A notable distinction being statically typed languages tend to enforce at compile-time that abstract methods be implemented in an inheriting class such that LSP doesn't end up being violated, whereas dynamically typed languages might have their type systems enforce a nominal hierarchy via annotations but this does not necessarily imply that LSP is met for all properties. The basic takeaway from this is that we can in a sense interpret type systems as existing on two spectrums, the extent to which they are integrated and applied within the languages runtime, and the strictness of the invariants it enforces.

3.3.2 Substitutability in Julia and C++. What we see in Julia is that it enforces nominal subtyping of abstract types in only the most direct sense, which is to say for a type S and an abstract type T as long as there exists a declared subtyping relationship $S <: T$ then we can use S in place of T . The declaration has no enforcement on the actual underlying structure of the object, here Julia falls back to simple error checks if a property access does not resolve. More broadly we can say that within Julia subtyping enforcements are entirely a function of the types declaration as opposed to any structural qualities of the object. We can see this in more rigorous terms through the work of Zappa et. al. [74] which aimed to formalize Julias implementation of subtyping.

Definition 3.13 (Julia Subtyping). As defined by Zappa et. al. we can denote the type level representation of a user-defined type as follows

$$\text{attr } \text{name}\{t_1 <: T <: t'_1, \dots, t_m <: T <: t'_m\} <: t'' \in \text{tds} \quad (15)$$

Here *attr* expresses if a type is concrete or abstract, *name* is the name of the type. Following this we have the type parameters T each which have their defined lower bound t_i and upper bound t'_i , in addition to any type bounds applied to the type itself t'' . The type definition is then a part of global type declarations *tds*. The inference rule for application of subtyping is then defined as follows

for a given typing context Γ

$$\begin{array}{c} \text{APP-SUPER} \\ \frac{\text{name}\{T_1, \dots, T_m\} <: t'' \in \text{tds} \quad \Gamma \vdash t''[a_1/T_1 \dots a_m/T_m] <: t' \vdash \Gamma'}{\Gamma \vdash \text{name}\{a_1, \dots, a_m\} <: t' \vdash \Gamma'} \\ \\ \text{APP-INV} \\ \frac{n \leq m, \Gamma_0 = \text{add}(\text{Barrier}, \Gamma), \forall i \in [0, n] \quad \Gamma_{i-1} \vdash a_i <: a'_i \vdash \Gamma_i \wedge \Gamma_i \vdash a'_i <: a_i \vdash \Gamma_i}{\Gamma \vdash \text{name}\{a_1, \dots, a_n\} <: \text{name}\{a'_1, \dots, a'_n\} \vdash \text{del}(\text{Barrier}, \Gamma_n)} \end{array}$$

Here the rule APP-SUPER expresses the application of a subtype to a supertype, the rule APP-INV expresses the application of a subtype to an inverse subtype. APP-INV introduce some new notation, namely *add* and *del* of a 'Barrier'; a 'Barrier' represents a tag which is used to delinate or distinguish between type variable introduced before and after the constructor. The check itself then simply ensures the constructor application is invariant.

The case of C++ is somewhat more complex, as its a statically typed language aside from enforcing a nominal hierarchy like Julia it also enforces a structural hierarchy so as to not violate LSP ahead of time. Informally we can say that given a type S and an abstract class of type T , for S to inherit from type T it must implement all the abstract methods of T , in a sense enforcing that its a valid substitution. A key difference from this to structural subtyping or some variants of parameterized structural subtyping is that while in the abstract context we loose any excess properties of the object. Stated differently inheritance based subtyping overlaps with parameterized structural subtyping if it enforces a concrete implementation of abstract methods, with the difference being we loose trivial access to excess properties (i.e. we need downcasting). To speak about this in more rigorous terms one approach we can take is first introducing the formalisms of structural subtyping and then seeing how inheritance based subtyping fits into this. If we consider a function in a dynamically typed language taking an object o and applying some set of behaviors which we can group as being described by some hypothetical type T . As structural subtyping only requires that an object passed as o only exhibits at least the properties of T we can imagine passing a bunch of objects which meet the requirements but also might have additional properties, if we construct a type T' which consists of the requires behaviors of T and the additional properties, then we can understand a structural subtype as being a type S which we can represent in T' as a composition of T and the additional properties of S . To express this formally we first introduce the idea of the type of all propositions and then build on this to define subtyping.

Definition 3.14 (Type of all propositions). The type of all propositions for a type S denoted Prop or Ω expresses a set of elements which are characterized by propositions which are true for S . In a simpler sense we can think of the type of all propositions as being the set of all possible descriptions about a part of the whole of S which are true. For the most general definition we say that Ω is a type with a type family T such that for all propositions $P : \Omega$ the type $T(P)$ is inhabited, which is to say that it has a single member

which acts as a proof of the proposition asserting that it is true.

$$\Omega : \prod_{P:\Omega} \prod_{a:T(P)} \prod_{b:T(P)} a =_{T(P)} b \quad (16)$$

The idea of subtyping then somewhat naturally follows from the idea of having a type subtype S of a type T if this type corresponds to some proposition in the set of all propositions of T . We can then define subtyping as follows.

Definition 3.15 (Subtyping). Given a type S and a type T having a type of all propositions Ω we can define a subtype of T as being a member p of the function type $P : S \rightarrow \Omega$ expressing a predicate which characterizes a subset of T i.e. the subtype. So we can say that

$$S \equiv \sum_{x:T} \text{El}(p(x)) \vdash S <: T \quad (17)$$

The notation $\text{El}(p(x))$ in this instance returns the element of the type family T which is inhabited by the proposition $p(x)$, in other words it returns x if $p(x)$ is true.

We can then elaborate and apply the general definition of subtyping to better understand the dynamics of structural subtyping. As a reminder we consider a valid structural subtype as a type which conforms to the required properties, and has additional properties. Hence we can think of an implicit structural supertype as being the type T whose elements all have the required properties and then some arbitrary combination of excess properties. Keeping this in mind we can then define the structural subtype as follows.

Definition 3.16 (Structural Subtyping). A structural subtype S of a type T is a type with an injection / embedding $i : S \hookrightarrow T$, which is to say that S can be represent as a subset of T . The predicate which asserts this quality can be expressed as follows

$$x : T \vdash \sum_{y:S} i(y) =_T x \quad (18)$$

The analogy to structural subtyping in dynamically typed languages here being again that T represents our implicit structural supertype and substitutability is determined if this type exists within the supertype T . It should be clarified that this is not an expression of any type checking mechanism but rather an analogous formalism for how we can think about the dynamics of structural subtyping.

Having this defined lets finally approach how we can express inheritance based subtyping in the context of structural subtyping. As a reminder with inheritance based subtyping we loose any excess properties, expressing this in terms of structural subtyping the analogy would be that we collapse all valid structural subtypes into their intersection as this by definition resolves to the required properties. If we think of the abstract properties as an index type into a family of subtypes then we can express the intersection of these subtypes as follows.

Definition 3.17 (I-Indexed Intersection). Given a type T , an index type I , and a family of structural subtypes $i : I \vdash S_i <: T$ with a family of embeddings $i_S : \prod_{i:I} S_i \hookrightarrow T$, we can define the intersection of these subtypes as being those members of S_i for which there exists a corresponding member in T .

$$\bigcap_{i:I} S(i) = \sum_{x:T} \prod_{i:I} \sum_{y:S(i)} i_S(i, y) =_T x \quad (19)$$

We can see that this intersection, is representative of a non-concrete subtype of T , with its concrete counterpart being represented by the depent product $\prod_{i:I} S(i)$. A valid downcast can then be understood as a function which takes the unified representation i.e. the intersection and returns the concrete representation. Presented informally as an inference rule we could express this as follows.

$$\frac{\text{DOWNCAST} \quad \Gamma \vdash i : I \vdash S_i <: T \vdash \bigcap_{i:I} S(i)}{\Gamma, \prod_{i:I} S(i) <: T}$$

Firstly it should be stated again that this is obviously a very abstract way of thinking about subtyping, but it offers some unique insights into how at least from a birdseye view certain attributes of type sytems only have very subtle differences in actuality but these differences can have a considerable impact on the actual practical day to day usage of a programming language. In this case the key takeaway is that inheritance based subtyping which enforces concrete implementations of abstract methods can be understood as a special case of structural subtyping in which we collapse all valid structural subtypes into their intersection which forms the abstract representation.

3.3.3 Representing Fields & Types. Given the above formalisms we can now approach how we can extend the library to support abstract types. We can work in a kind of top down fashion starting from the fundamental type representation to the actual bridging code. To bring us back to where we started we had the most basic form of our record represented as a type family indexed by a set of labels. We had 3 main components to this, the the label, the object type, and the dependent field type. The corresponding representation in C++ can then be seen in the template parameter list for the P template metafunction in listing 17.

src/cpp/usertype.cpp

```
1  template <typename Ot, typename Ft, const char* name>
2  struct P {
3      using obj_t = const Ot;
4      using field_t = Ft;
5      static constexpr const char* get_name() {
6          return name;
7      }
8      static std::function<Ft(Ot&)> get;
9      static std::function<void(Ot&, Ft)> set;
10 };
```

Listing 17: Type meta-function

As a reminder, contrary to a typical function, a template meta-function is one which takes in a set of statically bound types as its arguments and returns a type as its result. The parameter list in this case is analogous to the base type component of the lens we developed in 14. The `obj_t` and `field_t` types are just trivial access mechanisms which reflect the parameterized type back. The function `get_name` is required to access the field name from a static context. Finally we have the getter and setter methods, similarly to the name reflection these are declared as static methods. The only distinction in our case from the traditional implementation of

a lens is that we do not return the updated object when setting a field as its just not necessary for our purposes.

To contrast this with the original implementation here instead of binding the field properties to the context of the `add_property` function we isolate them as an entirely static metafunction. There are a few benefits to this, but the core motivation being to simply create a further separation between how we specify a type and the tools to construct said type, and the actual logic applying these tools. Another benefit of this is that, while currently there exists no native reflection within C++ it should be possible to replace the manual specification of these metafunctions with automated ones with the full addition of reflection.

A full type, or more concretely in our case object is then a composition of these fields. As metafunctions usually act on the type level as opposed to the value level we pass them to functions as template arguments. Here we have variadic templates come into play which allow us to pass an arbitrary number of our metafunctions to a function, in turn allowing us to construct this composition of fields. The original implementation divided this process of defining types into two parts, the C++ sided definition i.e. the type registration through the `add_property` function, and the Julia sided definition i.e. the type implementation and mirroring. As I wanted to generalize the field or property definition as a function which registers a type over a group of fields as opposed to for each field individually I adapted the logic into the `initialize_type` function, the signature of which can be seen in listing 18.

```
libs/jluna/include/usertype.hpp

1  template<
2      typename... P, template <typename...> class A,
3      typename... D_t, template <typename...> class B
4  >
5  static void initialize_type(A<P...>, B<D_t...>);
6
7  template <typename... D_t>
8  static void initialize_map();
```

Listing 18: Type initialization signature

One caveat we encounter when working with variadic functions is how to delineate between distinct sets of arguments. In our case we needed to not only pass the set of fields as P but we also needed to pass the derived types relevant for a particular object to preform the necessary downcasts to loosing excess properties known. To delineate between these two sets of arguments we define a variadic container as being a metafunction which simply acts a means of enclosing a set of types. In our case we can see these lists as the parameters `<typename...> class Name`, here we pass these lists as value level metafunctions which then allows us to define two distinct sets of type lists P and D_t with D_t representing our derived types. At compile-time this merely acts as a way of specializing our type registration for the given specification, at runtime this specialization is then used to actually register all of the types for the C++ sided code.

The Julia sided type mirroring happens on the C++ through the implementation declaration. We can see the signature of this in the listing 19. The most relevant parts here are the two type parameters

T referring to the type of the object and U being the supertype. On a high level we can think of this expressing to Julia that within a context of a module (the default just being the global scope) we want to establish the relationship $T <: U$. The type generation triggered through this call is then executed at runtime and calls into the Julia sided portion of the library to actually create a type with this relationship.

```
libs/jluna/include/usertype.hpp

1  template<typename T>
2  template<typename U>
3  void Usertype<T>::implement(unsafe::Module* module)
```

Listing 19: Type implementation signature

How we then use both this C++ sided initialization and its corresponding runtime registration and the Julia sided mirroring is shown in an example with listing 20. Here `t1` represents our basic type list containing the properties of the sphere type, the second argument is the relevant derived types for the properties, in this instance just the lambertian material. The implementation then expresses the relationship `sphere <: Hittable` and at runtime uses the information from the registration to mirror the type with the given properties.

```
src/cpp/usertype.cpp

1  Usertype<Sphere>::initialize_type(
2      TList<
3          Lens<center, &Sphere::center>,
4          Lens<radius, &Sphere::radius>,
5          Lens<r_squared, &Sphere::r_squared>,
6          Lens<mat, &Sphere::mat>,
7          Lens<bbox, &Sphere::bbox>
8      >(),
9      TList<Lambertian>()
10 );
11
12 Usertype<Sphere>::implement<Hittable>();
```

Listing 20: User type example application

3.3.4 Semantics of Unboxing & Boxing. The final and most major part of the implementation was extending the actual bridging process, both by generalizing the logic to work over objects as opposed to fields and by accounting for how to handle abstract types. Before delving into any of the specifics we have to first introduce a final mechanism in C++ which was heavily used within this process, this being the *fold expression*. As the name implies this mechanism enables the folding of a set of expressions into a single expression through some binary operation. In C++ this is divided into two forms of folding, binary and unary with there being a further subdivision into a left fold and a right fold. The relevance of this mechanism to our extension is that since we are generalizing our process to work over a template pack as opposed to just single type arguments we need a way to work with this pack. In our case

we use a unary fold with a ‘`,`’ operator. Per the specification an instantiation of a left unary fold expression is expressed as follows

$$(\dots op E) \rightarrow (((E_1 op E_2) op \dots) op E_n) \quad (20)$$

The comma operator can be understood in the context of a fold expression over a template pack as a way of chaining together or sequencing an expression over a set of types. So if we think of our expression E as being parameterized by a set of types T_i then the fold expression would simply be a way of applying E to each T_i in the pack. Expressed using the notation in 20 we could present this as follows.

$$(\dots, E\langle T_i \rangle) \rightarrow (((E\langle T_1 \rangle, E\langle T_2 \rangle, \dots), E\langle T_n \rangle)) \quad (21)$$

Starting with the unboxing process we can remind ourselves that the core issue here was that the unboxing mechanism we wanted to use for a field of an abstract type had to correspond to the underlying concrete type given the general design of the library and resulting constraints. The key aspect to the solution in the unboxing case is that, since Julia uses structural subtyping fundamentally, we know always know the concrete type from the Julia-sided end, thus we can trivially derive the concrete unboxing mechanism within the library if we create a correspondence between a pair consisting of a field label and a valid subtype of this field to the unboxing mechanism for this subtype and field. More practically we can implement this mapping as follows

```
libs/jluna/include/usertype.hpp
1 static inline std::unordered_map<
2     std::pair<std::string, std::string>,
3     std::function<void(T&, unsafe::Value*)>
4 > _type_dispatch_map = {};
```

Listing 21: Type dispatch map

The resulting unboxing mechanism is then a simple lookup into this map. The actual instantiation of this map then case two main cases, fields with concrete types and field with abstract types. If a field has a concrete type we want to create the trivial mapping of concrete type to concrete setter, if the field has an abstract type we want to resolve all valid derived types and then create a set of mappings to all valid derived types. In simple terms we create a map consisting of all valid concrete instantiations for all fields of an object. One notable feature which will play a role in the boxing process aswell is the type filtering mechanism. An early implementation design choice was that in the type specification all derived types relevant to a particular object would be passed as a grouped template pack, a benefit of this being that with manual specification we avoid forcing repeated specifications, so if two fields for example have the same abstract type we only need to specify the derived types once. Furthermore we reduce implementation complexity somewhat as the set of types doesnt exist as a member of a Lens but instead as just an auxillary pack. A key downside to this is that naturally we dont have a direct reference of what field Lens has what relevant derived types thus, if we iterate over the fields, to avoid instantiating invalid setters we have to first filter out derived types which are irrelevant for a certain field. In simple terms, if a field for

example has a concrete type A then we dont want to instantiate setters for any derived types irrelevant to A . To avoid this process a type-level filtering mechanism was implemented which can be seen in listing 22.

```
libs/jluna/.src/usertype.inl
1 for_each<FilterType<
2     FieldType, is_base_of, DerivedType...
3 >>>();
```

Listing 22: Type filter

The mechanism is for the most part pretty simple, it consists of an initial type T , a binary type predicate P and a set of types T , the resulting filtered type set T_f is then the set of types T for which the predicate $P(T_1, T_2)$ is true. There are various points to be made in regards to the pros and cons of this approach, the primary motivation in this case was to simplify the type specification and relegate the actual filtering process to the compiler. If we consider the automated case of parsing types then this mechanism might have less utility as we could implicitly construct the valid type specification as a part of the parsing algorithm itself, though it still does increase the complexity of the implementation somewhat regardless, as an implicit valid specification would mean the implementation does not act linearly but rather on a tree of types from the get go. Ultimately whats important to consider is the tradeoff in implementation complexity, compilation time (specifically template instantiation) and code bloat (i.e. limited the amount of code generated). Finally we have our unboxing function for usertypes which can be seen in listing 23.

```
libs/jluna/.src/usertype.inl
1 template <typename T>
2 T Usertype<T>::unbox(unsafe::Value* in) {
3     if (not _implemented) [[unlikely]]
4         implement();
5
6     gc_safe_point(
7         T out;
8         _unbox(out, in)
9     )
10    return out;
11 }
```

Listing 23: Unboxing usertype

Here `_unbox` is the call to the map lookup, `gc_safe_point` is a macro that preserves the julia-sided input `in` and `out` is the output of the unboxing process.

Similarly to the case of unboxing the boxing process requires that we know the underlying concrete type so we can use the correct mechanism to map to the julia sided type. The key difference here is that unlike in the case of Julia we do not trivially know the correct subtype given an abstract type in C++. The solution is then unfortunately a brute force one, we have to iterate over all valid derived types for a given abstract field and then box the derived

type which has a valid downcast. There are a few optimization points to consider here, again similar to the type dispatch map we filter out the set of valid derived types for a given field to not perform unnecessary downcast checks if we are boxing a concrete field. Furthermore we know that if a dynamic cast succeeds then we want to shortcut out cast checks for the remaining derived types. A final somewhat nuanced point to consider is that we also don't want to try to downcast a concrete type, so we have to ensure that for the case of a trivial boxing we preferably want to immediately box the concrete type without any additional runtime checks. We can achieve all of this through a combination of approaches. On a high level we can implement this as follows.

On a high level there are 3 main components to the boxing process, the input; which we access via the generic Field getter; the type switch statement MatchTy which specializes, or matches, on a particular generic lambda given a certain type predicate, and the finalizer which is a simple lambda that boxes the value we get from the matching process. The process of boxing is then roughly as follows, we use a fold expression to iterate over the set of fields, for each field we use a custom parameterized fold expression which discards any invalid derived types at compile time through the compile time type filtering mechanism. In the case of a concrete type we simply box the value, return it, and set it using the finalizer. If the value does have derived types, the fold expression internally iterates over the set of valid derived types and checks if a value is a valid downcast, to avoid unnecessary checks once we have a valid downcast we short circuit the fold expression.

3.4 Evaluation

We can assess the performance of the library in a few different ways, we can look at the performance, the developer experience and how extensible and maintainable this degree of type level programming is, and finally we can assess the user experience of the library.

3.4.1 Performance and Optimizations. Overhead was the main performance metric that was evaluated for the library, as its generally the most critical factor in the performance of language interoperability especially in the applied case of a raytracer where we want to ensure as minimal overhead as possible to be able to take advantage of the performance benefits of C++. To measure the overhead each raycast had its time measured starting at the julia sided implementation calling the C++ sided code, then in the C++ sided code before we traverse the BVH tree, and finally at the end of the C++ sided code and once we return to the julia sided code. In simple terms the overhead for unboxing and boxing was computed as seen below.

$$\text{unboxing time} = \text{cpp}_1 - \text{julia}_1 \quad (22)$$

$$\text{boxing time} = \text{julia}_2 - \text{cpp}_2 \quad (23)$$

A noticeable impact on performance noted during the optimization process was the lack of type stable code in the julia sided portion of the library. Once this was resolved by adding type annotations which minimally constrained the implementation the performance improved considerably. Another optimization applied on the C++ end was based on the observation that to call the correct boxing

or unboxing function for a given field the original implementation generated a map via a fold expression which was then used to lookup the correct function given the field. To get around the possible overhead from the map lookup the boxing and unboxing logic were turned into inlined isolated lambdas which were then called directly, and implemented such that they would generate the minimal necessary code to box/unbox the object as directly as possible to minimize any unnecessary intermediate calls.

We can observe the performance over the different raycasts in figure 9. Some notable observations to make, both the boxing and unboxing overhead are relatively stable towards the start and end of the execution, this represents the passes where a ray did not hit anything. When the ray did hit something we can notice spikes in the overhead, especially the unboxing as we need to update the hit record on the C++ then julia sided end to accurately render the pixel. One somewhat elusive observation is the consistent spike in the boxing process, due to the non-trivial nature of the entire pipeline this could be caused by various factors, though seems to be overall not too impactful on the average runtime.

Metric	Average (µs)	Total (µs)
Unboxing	128.459	12,554,152.684
Boxing	6.010	587,352.831
Actual	0.662	64,665.907
Overhead	134.469	13,141,505.515

Table 3: Summary of performance metrics

Observing a summary of the metrics in 3 we can notice that despite the optimizations the overhead is still quite considerable in our example. Naturally as we are working with more complex datastructures; as this is the primary target of the exploration; the interoperability overhead will be more pronounced. It's difficult to assess the broader implications of these performance metrics its quite clear that the overhead in this instance undoes any possible performance gains we could have from the C++ sided code. This is not to say that the library is entirely useless, a quite effective usage of gluecode is in migration of certain codebases to another language or in the case where a library may not be performance critical. A main takeaway from this is that while we are able to achieve idiomatic interoperability between two languages in our applied task, its application must broadly speaking be carefully considered and benchmarked as to assess if the overhead is worth the performance gains.

3.4.2 Developer Experience. One aspect the original implementation had in mind which I tried to continue with my extension was utilizing modern C++ features to improve the developer experience of working with the code. The main aspects of this being the use of well documented and type safe template metaprogramming in addition to functional programming and more broadly declarative programming paradigms. What this meant in practice is that I tried to design the implementation, especially the more abstract portions of the code, such that they would clearly reflect the most critical parts of the implementation with as little noise as possible. Templates are somewhat notoriously well known for being complex to read, to mitigate this I followed the style of the original implementation in keeping heavy template or type predicate code separate from

```

jluna/usertype.inl

1 _box = [&](T& in, unsafe::Value* out) {
2     (fold<FilterType<FType<Field>, std::is_base_of, DerivedType...>>(
3         Field::getter(in),
4         [&](auto&& val) {
5             unsafe::call(setfield, out, _fsymbols[Field::label()], val.value());
6         },
7         MatchTy {
8             // Boxable type
9             [&]<typename DT = FType<Field>, is_boxable FT>(FT& v) -> unsafe::Value* {
10                 return jluna::box<FT>(v);
11             },
12             // Not boxable, but base type
13             [&]<is_base_of<FType<Field>> DT, is_not_boxable FT>(FT& v) -> unsafe::Value* {
14                 if (auto* val = dynamic_cast<DT*>(v.get()))
15                     return jluna::box<DT>(*val);
16                 return nullptr;
17             }
18         }
19     ), ...);
20 };

```

Figure 8: Boxing usertype

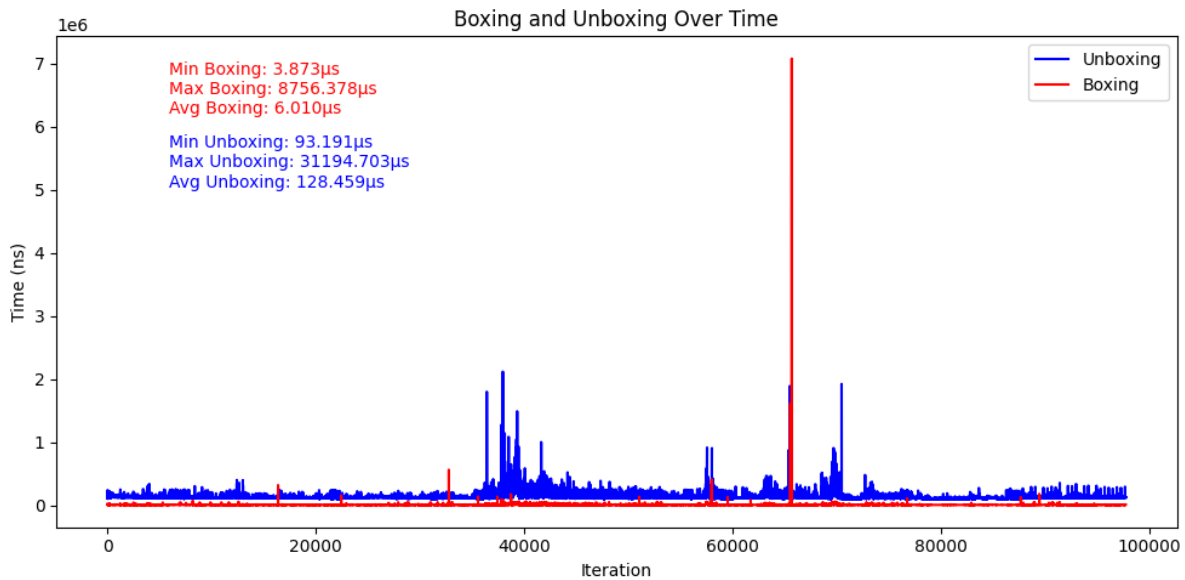


Figure 9: Boxing and unboxing overhead over time; measured on a system with Intel Core i7-1065G7 and 16GB RAM

the main implementation, it was also ensured that types have clear and descriptive names. Functional or more complex compile time type constructs were integrated in a way to reflect a sort of 2 level language where abstractions of templates are used in a way more naturally akin to a functional compile-time programming language separate from the runtime implementation and with the underlying type logic being separated from the actual implementation.

3.4.3 User Experience. The user experience is somewhat multifaceted depending on the priorities of the user. while the library is not on the level of having fully implicit interoperability it does provide a relatively simple interface to declare types and functions with

it while in the current state this is a somewhat mundane action it could be automated with relative ease once reflection is introduced in C++26. A key benefit of this library is that despite requiring some explicit declarations these are largely separate from the main implementation, this isolation allows for more versatility in the application of the library as opposed to ones which require a tighter coupling between the interoperability code and the main implementations in either languages. As mentioned before the performance overhead is something that requires careful consideration when using the library, to aid in this a basic userfacing benchmarking tool was implemented for the C++ side, and julia being a dynamic

high-level language has various tools to trivially benchmark from its end.

3.5 Limitations

The limitations of this extension and more broadly the idea of expanding interoperability libraries to support more complex types can be understood by assessing first some important language level considerations and then looking at the broader landscape of the evolution of programming languages and their actual real-world applications.

3.5.1 Slow and varying generics support. Generics support or the more complex type level programming is a key aspect of building more implicit idiomatic interoperability libraries that aren't reliant on external scripts or tools to parse and generate the necessary code. C++ and Julia are both languages which have in their own ways developed considerably complex type systems, a benefit of this focus in their design is that we can leverage these systems especially on the C++ side end to approach the problem of handling many different types in highly generic ways to minimize the amount of boilerplate code we need to write. Treating the problem of interoperability with this type level approach while quite interesting in this case could pose a significantly larger challenge in languages which don't have as developed type systems. Furthermore it's not even a guarantee if languages will continue to develop increasingly complex type systems as even general purpose programming languages often end up having domain specific requirements or design choices which can deliberately limit a highly expressive type system. A type-level approach might then be a good approach in languages that easily support it, but could be detrimental in languages which are based on incompatible design philosophies.

3.5.2 Asymmetric metaprogramming features. As was briefly touched upon in the previous section, due to this varying development and design of different programming languages they often have very different metaprogramming features. As our implementation is heavily reliant on the metaprogramming features for things like, type level functional logic, reflection, type generation, specialization, and so on, similarly as to the previous point it could be difficult to use these design choices as a more general framework for interoperability systems. Metaprogramming does exist to various degrees in many languages but its featureset should be closely evaluated to see if it's sufficient to use as the basis for an interop. library or if other approaches would be more suitable.

3.5.3 Decentralized language evolution. If we look at the broader landscape of programming languages we can see that a general trend is that languages tend to evolve within their own ecosystems. Naturally there are exceptions to this trend, and there have been various attempts to build languages and more abstractly frameworks that enable various different languages to exist within the same ecosystem. A key example of this being the JVM or CLR, and more recently the GraalVM, but we can observe that despite these developments many of the mainstream popular languages seem to evolve within their own community and ecosystem. This is not to say that these languages are entirely isolated, we do often see theoretical or practical aspects of their implementation being inspired by other languages or built on existing tools, a prime example of this

being the set of languages developed on LLVM. Even with the often theoretical or technological overlap of languages a limiting factor in creating traditional interop libraries that enable the bridging of more complex properties of languages, especially ones that diverge in their design philosophies is that the languages are often inherently not designed to interoperate as we often see them created and evolve to address either a deficit in another language or some completely different domain specific niche. We have seen various developments to bridge separate language ecosystems such as WASM or the C-ABI, but these are often limited and as we established come with certain fundamental drawbacks. We have also observed that languages which are built on an inherently unifying framework are able to overcome a lot of the deficits of more traditional interop libraries, but the existing generally highly well established language ecosystems are continuing to often be the primary means by which people interact and work with these languages.

3.5.4 Theoretical exploration vs practical needs. A final point to consider is the basic reality or question of "is interoperability past basic constructs even necessary in the first place?". In an industry setting and especially at larger scales it's a somewhat natural byproduct of the environment to focus on the more short-mid term outcomes of a piece of software, what this tends to lead to is a preference for easy of use, safety, reliability, and so on. An observation that can be made from this preference for safe, reliable and easy software in a practical setting is that it often rarely attempts to evolve beyond what is actually needed and what can be reliably trusted. Legacy systems and codebases tend to be the natural result of this preference, and while this is not inherently a bad thing, it does in many instances mean that as opposed to spending a lot of time exploring or ruminating on what latest features in a language could be used the focus is more on simply working with what you have to get the job done. Exploration and being more open to newer technologies tends to be more common in either a research setting or sometimes only even possible in instances where something is fiscally viable. Bridging more complex language features while interesting to then explore in a theoretical sense is then not necessarily a real practical need for many developers. Research and academia being ahead of what is used in day to day life is not a new phenomenon, but in this instance there is quite simply more research needed to assess if this is a more broadly applicable approach to interoperability given the constraints elaborated on above.

4 CONCLUSION

Understanding the modern state of language interoperability is a complex task. Everything from the lowest level of the programming stack to the highest level of abstractions plays a role in well languages play with one another. Despite this complexity there is a structure to it that can be understood, and furthermore utilized to better understand how interoperability tools have evolved and likely will evolve. The goal of this paper was to build a framework for understanding the evolution of more traditionally based interoperability tools, and to build on this framework - and an existing tool - to explore more complex, niche and possible future applications and assess their broader viability. The primary insight of this paper is that FFI based tools despite their long history still have considerable relevance and room to evolve further in the future. A

secondary insight is that FFI and more unified execution models and their associated interoperability tools are not entirely mutually exclusive, which is to say that alot the more recent theoretical foundation of unified execution models can be applied to FFI based tools to improve their performance and usability. Ultimately its hard to gauge how things will evolve in the future, but its quite evident that language interoperability is a complex and multi-faceted problem deeply tied into the evolution of programming languages.

REFERENCES

- [1] D. Abrahams and S. Seefeld, *Boost.python library documentation*, Version 1.72.0, Accessed: 2025-06-01, 2019. [Online]. Available: https://www.boost.org/doc/libs/1_72_0/libs/python/doc/html/index.html.
- [2] S. Akbary, *Wasienv: Wasi development workflow for humans*, <https://blog.wasmer.io/wasienv-wasi-development-workflow-for-humans-1811d9a50345>, Accessed: 2025-05-27, 2019.
- [3] S. Akbary, M. McCaskey, J. Kruth, et al., *Wasienv/wasienv*, Jun. 6, 2023. [Online]. Available: <https://github.com/wasienv/wasienv>.
- [4] Armin Rigo and contributors, *Cffi documentation*, C Foreign Function Interface for Python, 2024. [Online]. Available: <https://cffi.readthedocs.io/en/latest/> (visited on May 25, 2025).
- [5] S. Audet and Contributors, *Javacpp - the missing bridge between java and native c++*, GitHub repository, 2024. [Online]. Available: <https://github.com/bytedeco/javacpp>.
- [6] Ayke, R. Evans, D. Gryski, et al., *Tinygo-org/tinygo*, Nov. 14, 2024. [Online]. Available: <https://github.com/tinygo-org/tinygo>.
- [7] E. Barrett, C. F. Bolz, and L. Tratt, “Approaches to interpreter composition”, *Computer Languages Systems and Structures*, vol. 44, pp. 199–217, Dec. 2015, issn: 1477-8424. doi: [10.1016/j.cl.2015.03.001](https://doi.org/10.1016/j.cl.2015.03.001).
- [8] Bytecode Alliance, *Bytecode alliance projects*, Accessed: 2025-05-27, 2023. [Online]. Available: <https://bytecodealliance.org/projects>.
- [9] Bytecode Alliance, *Cargo-wasi: A lightweight cargo subcommand for the wasm32-wasi target*, Provides tools for building, running, testing, and benchmarking Rust code for the wasm32-wasi target. Licensed under Apache-2.0 with LLVM exception., 2024. [Online]. Available: <https://github.com/bytecodealliance/cargo-wasi>.
- [10] Bytecode Alliance, *Wasmtime documentation*, Wasmtime is a standalone runtime for WebAssembly, WASI, and the Component Model. It is designed to be fast, secure, configurable, and standards compliant, and can run WebAssembly outside of the web environment., 2024. [Online]. Available: <https://github.com/bytecodealliance/wasmtime>.
- [11] Bytecode Alliance, *Component model concepts*, <https://component-model.bytecodealliance.org/design/component-model-concepts.html>, Accessed: 2025-05-27, 2025.
- [12] C. Carruth, *Object relocation in terms of move plus destroy*, <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p1144r13.html>, ISO/IEC JTC1/SC22/WG21 P1144R13, 2025.
- [13] Clang Documentation, *Objective-c automatic reference counting (arc)*, Accessed: 2025-06-02, 2023. [Online]. Available: <https://clang.llvm.org/docs/AutomaticReferenceCounting.html>.
- [14] Clang Documentation Contributors, *Libclang - c interface to clang*, <https://clang.llvm.org/docs/LibClang.html>, Accessed: 2025-06-12, 2025.
- [15] Clang Team, *Clang attribute reference: trivial_abi*, <https://clang.llvm.org/docs/AttributeReference.html#trivial-abi>, Accessed: 2025-06-04, 2025.
- [16] J. contributors, *Garbage collection and javacall*, <https://juliainterop.github.io/JavaCall.jl/gc.html>, Accessed: 2025-06-05.
- [17] R. Cooper, “Records and record types in semantic theory”, en, *Journal of Logic and Computation*, vol. 15, no. 2, pp. 99–112, Apr. 1, 2005, issn: 0955-792X. doi: [10.1093/logcom/exi004](https://doi.org/10.1093/logcom/exi004). [Online]. Available: <http://dx.doi.org/10.1093/logcom/exi004>.
- [18] C. Cords, *Jluna: A modern julia wrapper for c++*, <https://www.clemens-cords.com/jluna>, v1.0.0, 2022. [Online]. Available: <https://www.github.com/clemapfel/jluna>.
- [19] cppreference.com, *C++ language: Class specifier*, Accessed: 2025-06-02, 2024. [Online]. Available: <https://en.cppreference.com/w/cpp/language/class>.
- [20] Crubit Contributors, *Struct layout*, https://github.com/google/crubit/blob/8d6b38fd9ab044ab7595a7df9752e62eb84345c7/docs/design/struct_layout.md, Accessed: 2025-06-04, 2023.
- [21] T. C. P. Developers, *Crubit design document: Cxx section*, <https://github.com/google/crubit/blob/main/docs/design/design.md#cxx>, Accessed: 2025-05-29, 2025.
- [22] T. J. Developers, *Libcxxwrap-julia: C++ library for backing cxxwrap.jl*, <https://github.com/JuliaInterop/libcxxwrap-julia>, Accessed: 2025-06-01, 2025.
- [23] DigitalOcean Community, *Java JVM memory model - memory management in java*, Accessed: 2025-06-05, 2020. [Online]. Available: <https://www.digitalocean.com/community/tutorials/java-jvm-memory-model-memory-management-in-java%7D>.
- [24] DutChen18, *leaningtech/ts2cpp*. Sep. 2, 2024. [Online]. Available: <https://github.com/leaningtech/ts2cpp>.
- [25] Emscripten Project, *File system overview — emscripten documentation*, Accessed: 2025-05-27, n.d. [Online]. Available: https://emscripten.org/docs/porting/files/file_systems_overview.html#file-system-overview.
- [26] W. S. Fulton, O. Betts, VZ, et al., *swig/swig*. Feb. 5, 2025. [Online]. Available: <https://github.com/swig/swig>.
- [27] Y. Futamura, “Partial computation of programs”, in *RIMS Symposia on Software Science and Engineering*. Springer Berlin Heidelberg, 1983, pp. 1–35, isbn: 9783540394426. doi: [10.1007/3-540-11980-9_13](https://doi.org/10.1007/3-540-11980-9_13).
- [28] O. Goffart, N. Layzell, twistedfall, et al., *mystor/rust-cpp*. Nov. 20, 2024. [Online]. Available: <https://github.com/mystor/rust-cpp>.
- [29] D. Gohman, A. Crichton, A. Brown, et al., *Webassembly/wasi-sdk*, Oct. 29, 2024. [Online]. Available: <https://github.com/WebAssembly/wasi-sdk>.

- [30] Google, *Crubit: C++/rust bidirectional interop tool*, GitHub repository, 2024. [Online]. Available: <https://github.com/google/crubit>.
- [31] Google and Contributors, *Cxx - safe interoperability between rust and c++*, Project documentation, 2024. [Online]. Available: <https://cxx.rs/>.
- [32] GraalVM Contributors, *Graalvm languages reference manual*, <https://www.graalvm.org/latest/reference-manual/languages/>, Accessed: 2025-05-27, 2025.
- [33] GraalVM Contributors, *Graalvm webassembly documentation*, <https://www.graalvm.org/webassembly/>, Accessed: 2025-05-27, 2025.
- [34] GraalVM Team, *Introduction to graalvm*, Accessed: 2025-05-25, 2025. [Online]. Available: <https://www.graalvm.org/latest/introduction/>.
- [35] W. Jakob, H. Schreiner, J. Rhineland, *et al.*, *Pybind/pybind11*, Nov. 14, 2024. [Online]. Available: <https://github.com/pybind/pybind11>.
- [36] B. Janssens, P. Jain, M. Horn, *et al.*, *JuliaInterop/CxxWrap.jl*, Jan. 29, 2025. [Online]. Available: <https://github.com/JuliaInterop/CxxWrap.jl>.
- [37] S. G. Johnson, T. Arakaki, Y. Yu, *et al.*, *JuliaPy/PyCall.jl*, Mar. 11, 2024. [Online]. Available: <https://github.com/JuliaPy/PyCall.jl>.
- [38] JuliaInterop, *Cxx.jl: The julia c++ interface*, <https://github.com/JuliaInterop/Cxx.jl>, Accessed: 2025-06-01, 2025.
- [39] H. Kalbasi, *Zngur: A c++/rust interop tool*, <https://github.com/HKalbasi/zngur>, Accessed: 2025-05-29, 2025.
- [40] H. Kalbasi, *Design decisions*, <https://hkalbasi.github.io/zngur/philosophy.html>, Accessed: 2025-05-31, 2025.
- [41] B. Kundu, V. Vassilev, and W. Lavrijsen, “Efficient and accurate automatic python bindings with cppy; cling”, 2023. doi: [10.48550/ARXIV.2304.02712](https://arxiv.org/abs/2304.02712). [Online]. Available: <https://arxiv.org/abs/2304.02712>.
- [42] J. Maurer, *Trivial relocatability*, <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p2786r11.html>, ISO/IEC JTC1/SC22/WG21 P2786R11, 2024.
- [43] Microsoft Corporation, *Common language runtime (clr) overview*, Accessed: 2025-05-25, 2023. [Online]. Available: <https://learn.microsoft.com/en-us/dotnet/standard/clr>.
- [44] Microsoft Learn, *Object lifetime and resource management (raii)*, Accessed: 2025-06-02, 2022. [Online]. Available: <https://learn.microsoft.com/en-us/cpp/cpp/object-lifetime-and-resource-management-modern-cpp?view=msvc-170>.
- [45] Office of the National Cyber Director, “Back to the building blocks: A path toward secure and measurable software”, The White House, Tech. Rep., Feb. 2024. Accessed: 2025-05-29. [Online]. Available: <https://bidenwhitehouse.archives.gov/wp-content/uploads/2024/02/Final-ONCD-Technical-Report.pdf>.
- [46] S. Pestov, *Library evolution in swift*, <https://swift.org/blog/library-evolution/>, Accessed: 2025-05-27, 2020.
- [47] S. H. Peter Shirley Trevor David Black, *Ray tracing in one weekend*, Aug. 2024. [Online]. Available: <https://raytracing.github.io/books/RayTracingInOneWeekend.html>.
- [48] S. H. Peter Shirley Trevor David Black, *Ray tracing: The next week*, Aug. 2024. [Online]. Available: <https://raytracing.github.io/books/RayTracingTheNextWeek.html>.
- [49] C. Piovesan, S. De Rossi, Alex, *et al.*, *leaningtech/cheerp-meta*, Mar. 29, 2025. [Online]. Available: <https://github.com/leaningtech/cheerp-meta>.
- [50] J. Rose, *Abi stability and more*, Accessed: 2025-05-25, Feb. 2019. [Online]. Available: <https://swift.org/blog/abi-stability-and-more/>.
- [51] C. Rowley, *Juliacall: Seamless integration of julia and python*, <https://pypi.org/project/juliacall/>, Version 0.9.25, released on May 14, 2025. Accessed: 2025-05-27, 2025.
- [52] C. Rowley, github-actions[bot], M. Cranmer, *et al.*, *JuliaPy/Python-Call.jl*, Jan. 22, 2025. [Online]. Available: <https://github.com/JuliaPy/PythonCall.jl>.
- [53] Rust Foundation, *Google contributes \$1m to rust foundation to support c++/rust “interop initiative”*, Accessed: 2025-05-29, Feb. 2024. [Online]. Available: <https://rustfoundation.org/media/google-contributes-1m-to-rust-foundation-to-support-c-rust-interop-initiative/%7D>.
- [54] A. Sengupta, M. Kittisopikul, B. Burdick, *et al.*, *JuliaInterop/JavaCall.jl*, Nov. 22, 2023. [Online]. Available: <https://github.com/JuliaInterop/JavaCall.jl>.
- [55] SingleStore Labs, *Wasix: Posix compatibility layer for wasi builds*, Provides stubs for POSIX compatibility in WASI builds, facilitating the compilation of sources expecting POSIX support. Designed for experimental use., 2024. [Online]. Available: <https://github.com/singlestore-labs/wasix>.
- [56] A. Taylor, P. Craig, B. Silverman, *et al.*, *google/autocxx*, Mar. 5, 2025. [Online]. Available: <https://github.com/google/autocxx>.
- [57] The Chromium Project Developers, *Rust and c++ interoperability*, <https://www.chromium.org/Home/chromium-security/memory-safety/rust-and-c-interoperability/>, Accessed: 2025-05-29, 2020.
- [58] The Julia Language, *Embedding julia*, Julia 1.x Documentation, 2024. [Online]. Available: <https://docs.julialang.org/en/v1/manual/embedding/> (visited on May 25, 2025).
- [59] The Julia Programming Language, *Just-in-time (jit) compilation — julia documentation*, Accessed: 2025-05-27, 2024. [Online]. Available: <https://docs.julialang.org/en/v1/devdocs/jit/>.
- [60] The Julia Project, *Julia documentation: The julia object model*, <https://docs.julialang.org/en/v1/devdocs/object/>, Accessed: 2025-05-31, 2024.
- [61] The Rust Project Developers, *Foreign function interface (ffi)*, Rustonomicon: Unsafe Rust Guidelines, 2024. [Online]. Available: <https://doc.rust-lang.org/nomicon/ffi.html> (visited on May 25, 2025).
- [62] The Rust Project Developers, *What is pinning?*, <https://doc.rust-lang.org/std/pin/index.html#what-is-pinning>, Accessed: 2025-06-04, 2025.
- [63] The rust-bindgen Developers, *Rust-bindgen*, <https://github.com/rust-lang/rust-bindgen>, Accessed: 2025-05-29, 2025.
- [64] D. Tolnay, *Cxx: Safe interop between rust and c++*, <https://github.com/dtolnay/cxx>, Accessed: 2025-05-29, 2025.

- [65] D. Tolnay, *syn: A parsing library for rust source code*, <https://docs.rs/syn/latest/syn/>, Latest version published on crates.io as of June 2025, 2025. [Online]. Available: <https://docs.rs/syn/latest/syn/>.
- [66] E. Unruh, *Prime number computation*, ANSI X3J16-94-0075 / ISO WG21-462, 1994. [Online]. Available: <https://stackoverflow.com/questions/48849447/does-example-of-erwin-unruh-work-today>.
- [67] V. Vassilev, A. Naumann, marsupial, *et al.*, *root-project/cling*. Mar. 7, 2025. [Online]. Available: <https://github.com/root-project/cling>.
- [68] T. L. Veldhuizen, *C++ templates as partial evaluation*, 1998. DOI: [10.48550/ARXIV.CS/9810010](https://arxiv.org/abs/10.48550/ARXIV.CS/9810010).
- [69] WASI Subgroup, W3C WebAssembly Community Group, *Wasi.dev: The webassembly system interface*, <https://wasi.dev/>, Accessed: 2025-05-27, 2025.
- [70] WasmEdge Contributors, *Build and run with ahead-of-time compilation*, <https://wasmedge.org/docs/start/build-and-run/aot/>, Accessed: 2025-05-27, 2025.
- [71] WebAssembly Community Group, *Webassembly features*, <https://webassembly.org/features/>, Accessed: 2025-05-27, 2025.
- [72] M. Young, silvanshade, and A. Taylor, *google/moveit*. Jun. 15, 2023. [Online]. Available: <https://github.com/google/moveit>.
- [73] A. Zakai, S. Clegg, juj, *et al.*, *emscripten-core/emscripten*. May 24, 2025. [Online]. Available: <https://github.com/emscripten-core/emscripten>.
- [74] F. Zappa Nardelli, J. Belyakova, A. Pelenitsyn, B. Chung, J. Bezanson, and J. Vitek, “Julia subtyping: A rational reconstruction”, en, *Proceedings of the ACM on Programming Languages*, vol. 2, no. OOPSLA, pp. 1–27, Oct. 24, 2018, ISSN: 2475-1421. DOI: [10.1145/3276483](https://doi.org/10.1145/3276483). [Online]. Available: <http://dx.doi.org/10.1145/3276483>.
- [75] E. Zhdan, “Swift / C++ Interoperability”, in *EuroLLVM Developers’ Meeting 2024*, Accessed: 2025-06-12, Apr. 2024.
- [76] S. Zheng, Z. Zhou, X. Chen, *et al.*, “Luisarender: A high-performance rendering framework with layered and unified interfaces on stream architectures”, *ACM Trans. Graph.*, vol. 41, no. 6, Nov. 2022, ISSN: 0730-0301. DOI: [10.1145/3550454.3555463](https://doi.org/10.1145/3550454.3555463). [Online]. Available: <https://doi.org/10.1145/3550454.3555463>.